导航

- »
-

Altair 8800 操作手册

Version

目录

目录

第 0 部分：前言：版本说明、致谢等 5

第 1 部分：数制、逻辑与程序设计 6

- A. 逻辑
- B. 电子逻辑
- C. 数制
- D. 二进制
- E. 八进制
- F. 计算机程序设计
- G. 一个简单程序
- H. 计算机语言

第 2 部分：ALTAIR 8800 的组成 21

- A. 中央处理器（CPU）
- B. 存储器
- C. 时钟
- D. 输入/输出

第 3 部分：ALTAIR 8800 的操作 26

- A. 前面板开关与 LED
- B. 载入示例程序
- C. 存储器的使用
- D. 存储器寻址
- E. 操作提示

第 4 部分：Altair 8800 指令集 38

- A. 命令指令
- B. 单寄存器指令
- C. 寄存器对指令
- D. 累加器循环移位指令
- E. 数据传送指令
- F. 立即数指令

- G. 转移指令

附录 A：ALTAIR 8800 指令集 90

- A. 命令指令
- B. 单寄存器指令
- C. 寄存器对指令
- D. 累加器循环移位指令
- E. 数据传送指令
- G. 转移指令

附录 B：在 Linux 上配置 Altair 8800 111

- 软件包
- 编写、汇编源代码
- 在软件模拟器中运行你的程序
- 配置 MINICOM
- ~/.MINIRC.ALTAIR
- 运行实测：在 Altair 克隆机上运行你的程序
- MAKEFILE
- MAKEFILE
- 获取输入 / 显示输出
- 提示
- Kermit 说明

补遗：快进到 2020 129

资源 134

服务 135

MITS ALTATR 8800 价目表 136

- 手册
- 邮费与条款

Altair 8800 操作手册

第 0 部分：前言：版本说明、致谢等

这项倾注热爱的工作由 Kevin Cole（"The Ubuntourist"）于 2020 年 9 月发起，旨在为 计算机系统课程 提供补充。

本文档的读者可以使用 Ian Davies 开发的出色 MITS Altair 8800 simulator 来测试自己的代码，或者只需 $621（US）即可拥有一台实体 Altair 8800 Clone，配有拨动开关、闪烁的红色 LED 等等。

初学者可能会觉得 YouTube 上 Patrick Jackson 的视频 相当有参考价值。

除了将 扫描印刷品 转换为可搜索的 HTML 之外，本版本还包含对原文档中排印错误的更正，以及一些补充材料，尤其涉及 半加器 电路、在 Linux 系统下汇编并加载代码的说明，以及 快进到 2020 年 — 后者同时包含用 **GNU Assembler（gas）** 和 **Netwide Assembler（nasm）** 编写的 **"Hello World"**。

若要深入了解 Intel 8080 CPU，请参阅 Intel 8080 编程手册，唉，它多半不会由我转换为 HTML 了。它有*一点* ☺ 太远了。

从 PDF 扫描印刷品到 HTML（以及最终更好的 PDF）的转换，是借助以下工具完成的：**pdftoppm**（poppler-utils 工具之一）、Tesseract-OCR、Inkscape、基于 Cloud 主题的 Sphinx，当然还有 `One True Editor`_（"顶礼膜拜！"）|wink|。

印刷品本身来自一座金矿：Classic Computing 网站上的 Dave Dunfield's 页面 Daves Old Computers。认真的学生和爱好者不妨戴上安全帽，去那里挖上一番。（金丝雀可选。☺）

源代码和渲染出的 HTML 托管在 **Codeberg.org**，因为我**真的**很喜欢他们的 使命宣言。

如果你发现了问题，可以提交 issue。

由于 Micro Instrumentation and Telemetry Systems（MITS）公司已经不复存在，我们曾尝试联系版权持有人，但可惜搜寻无果。

无论如何，原作的版权归 MITS, Inc 所有：Copyright © 1975。

第 1 部分：数制、逻辑与程序设计

半导体技术的卓越进步使 ALTAIR 8800 的开发成为可能：它是迄今为止最经济的计算机，也是第一种同时以套件和整机两种形式供应的计算机。

本操作手册旨在帮助新手和经验丰富的计算机用户熟悉 ALTAIR 8800 的操作。这台计算机有 78 条机器语言指令，能够执行若干传统小型计算机通常不具备的重要操作。读完本手册，即使是新手也能把程序装入 ALTAIR 8800 。

ALTAIR 8800 的用户中，既有电子学背景扎实但几乎没有计算机经验的人，也有相当编程经验但几乎没有电子学背景的人。因此，本手册的编写兼顾了所有类型的用户。

手册的 第 1 部分 帮助用户更好地理解计算机术语、技术和操作，内容包括传统逻辑与电子逻辑的引论、若干重要数制的说明、基础程序设计的讨论，以及计算机语言的论述。

手册的 第 2 部分 和 第 3 部分 描述 ALTAIR 8800 的组织和操作，重点放在计算机中用户最常使用的那些部分上。最后，手册的`第 4 部分`_ 详细列出 ALTAIR 8800 的 78 条指令，附录 则把这些指令压缩成一份快速参考清单。

即使你对计算机的操作和组织几乎没有经验，仔细阅读本手册也能使你学会操作 ALTAIR 8800 。随着你对这台机器经验的积累，你很快就会理解它真正令人难以置信的通用性和数据处理能力。如果本手册在某些地方显得太复杂，不要气馁。只要记住：计算机只做它的程序员指令它的事。

A. 逻辑

George Boole 是十九世纪英国数学家，他详细研究了某些基本逻辑表达式与其算术对应物之间的关系。布尔并不把数学等同于逻辑，但他确实展示了任何逻辑陈述都可以用简单的算术关系来分析。1847 年，布尔出版了一本名为 逻辑的数学分析 的小册子；1854 年，他就该主题出版了一部详尽得多的著作。直到今天，所有实用的数字计算机和许多其他电子电路都建立在布尔阐明的逻辑概念之上。

布尔的逻辑系统常被称为布尔代数，它假定一个逻辑条件或陈述要么为真，要么为假。它不可能既真又假，也不可能部分为真或部分为假。幸运的是，电子电路非常适合于这种双状态运行。如果把导通状态的电路称为真、断开状态的电路称为假，就可以方便地合成出逻辑陈述的电子类似物。

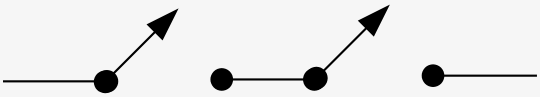
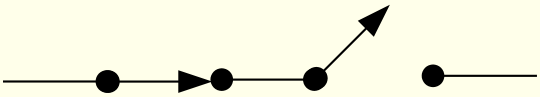
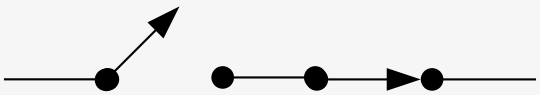
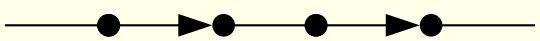
据此，可以为三种基本逻辑语句—与、或、非—设计电子等价物。仅当 所有 逻辑条件都为真时，与语句才为真。如果 任一 逻辑条件为真，或语句就为真。非语句只是颠倒逻辑语句的含义，使真语句变成假，假语句变成真。

用通断开关可以方便地生成这三种逻辑语句的简单等价物。接通的开关被称为真，断开的开关被称为假。由于断开的开关不让电流通过，可以给它赋数值 0；类似地，接通的开关能让电流通过，可以赋数值 1。

现在，通过考察双条件与语句的各种排列组合，我们可以设计出逻辑与语句的电子等价物：

条件（输入）	与	条件（输入）	结论（输出）
假	与	假	假
假	与	真	假
真	与	假	假
真	与	真	真

这些排列组合的电子开关等价形式很简单：

条件	结论
（关-开）	（输出）
	0
	0
	0
	1

类似地，这些排列组合的数值等价形式如下：

条件（输入）	与	条件（输入）	结论（输出）
0	与	0	0
0	与	1	0
1	与	0	0
1	与	1	1

数字设计工程师把这些排列表称为真值表。双条件与语句的真值表通常这样给出：

图 1-1. 与功能真值表

A	B	OUT
0	0	0
0	1	0
1	0	0
1	1	1

现在可以推导出或语句和非语句的真值表，它们分别如图 1-2 和图 1-3 所示。

图 1-2. 或功能真值表

A	B	OUT
0	0	0

A	B	OUT
0	1	1
1	0	1
1	1	1

图 1-3. 非功能真值表

A	OUT
0	1
1	0

B. 电子逻辑

三种基本逻辑功能都可以用相对简单的晶体管电路实现。按照惯例，每种电路都被赋予一个符号，以便于设计逻辑系统。这三个符号及其各自的真值表如图 1-4 所示。

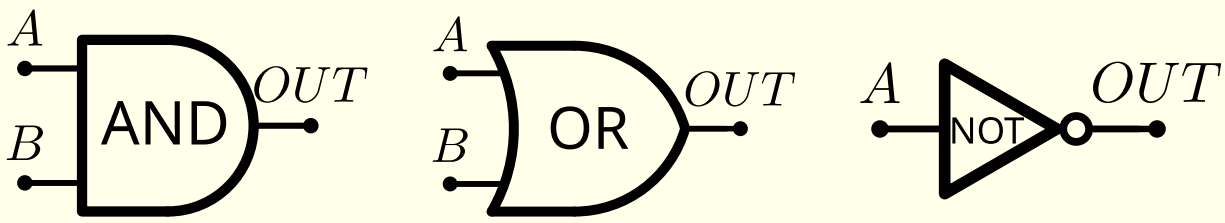


图 1-4. 三种主要逻辑符号

与门			或门			非门	
A	B	OUT	A	B	OUT	A	OUT
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

这三种基本逻辑电路可以相互组合，产生更多的逻辑语句模拟。这些电路组合中有两种使用得如此频繁，以至于被视为基本逻辑电路，并被赋予了各自的逻辑符号和真值表。这两种电路是与非门（NAND，即 NOT-AND）和或非门（NOR，即 NOT-OR）。图 1-5 给出了这些电路的逻辑符号和真值表。

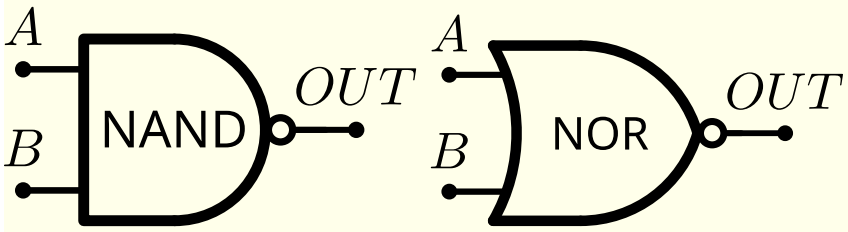


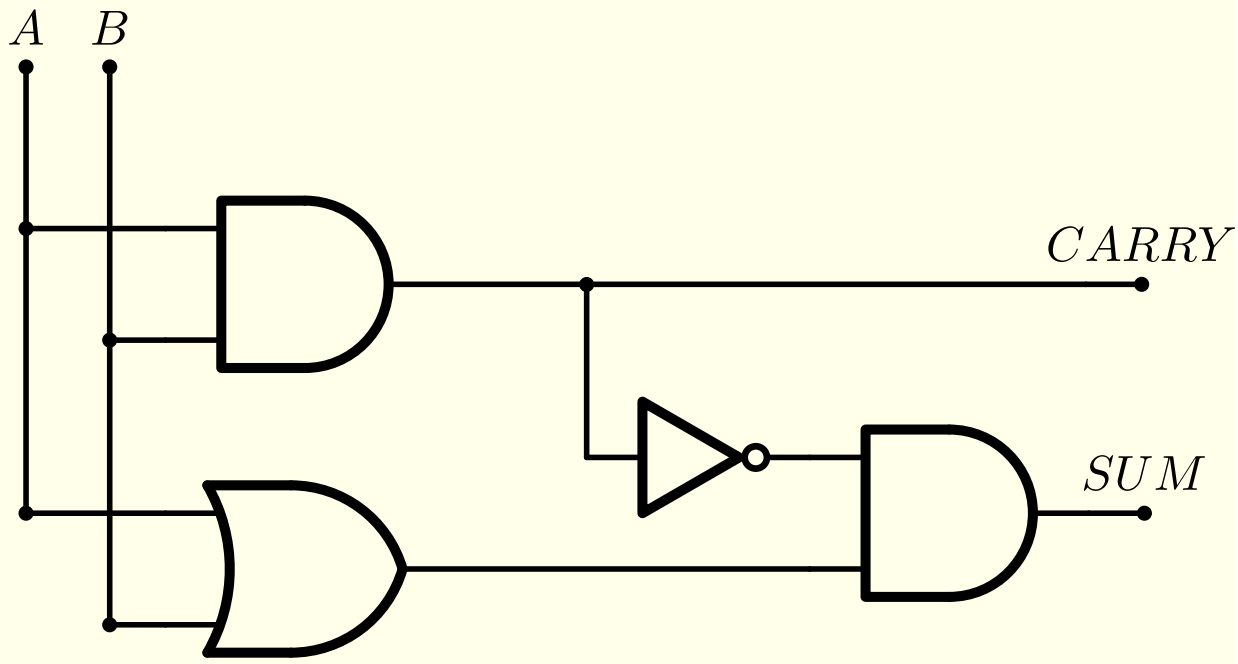
图 1-5. 与非门和或非门电路

与非门			或非门		
A	B	OUT	A	B	OUT
0	0	1	0	0	1
0	1	1	0	1	0
1	0	1	1	0	0

与非门			或非门		
A	B	OUT	A	B	OUT
1	1	0	1	1	0

三个或更多的逻辑电路构成一个逻辑系统。最基本的逻辑系统之一是如图1-6 所示的异或电路。

图 1-6. 异或电路



A	B	A 与 B	非（A 与 B）	A 或 B	（非（A 与 B）与（A 或 B））
		CARRY			OUT 即 SUM
0	0	0	1	0	0
0	1	0	1	1	1
1	0	0	1	1	1

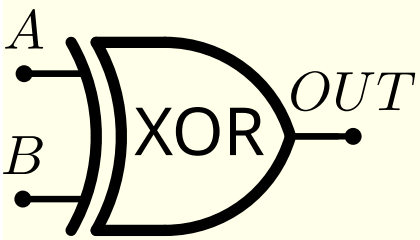
A	B	A 与 B	非（A 与 B）	A 或 B	（非（A 与 B）与（A 或 B））
		CARRY			OUT 即 SUM
1	1	1	0	1	0

异或电路可用于实现逻辑功能，但也可以用来把两个输入条件相加。由于电子逻辑电路只使用 0 和 1 两个数字单位，它们与只有两个数字的二进制数制兼容。因此，异或电路常被称为二进制加法器。

只看图 1-6 中的 CARRY 和 SUM 列，把它们合起来看作一个两位二进制数，得到：

A	B	A + B		
		CARRY	SUM	RESULT
0	0	0	0	00
0	1	0	1	01
1	0	0	1	01
1	1	1	0	10

通常，在提及异或（XOR）时，CARRY 被忽略，其图形表示为：



逻辑电路的各种组合可用于实现众多电子功能。例如，两个与非门电路可以连接起来，组成一种称为触发器的双稳态电路。由于触发器只有在脉冲形式的输入信号到达时才改变状态，它起着短期存储器元件的作用。若干触发器可以级联在一起，组成电子计数器和存储寄存器。

其他逻辑电路可以连接在一起，组成单稳态和无稳态电路。单稳态电路除非接收到输入脉冲，否则处于两个状态之一；随后它短暂地处于相反状态，然后恢复正常状态。无稳态电路持续地在两个状态之间来回切换。

C. 数制

早期人类可能因为发现用手指计数很方便，便发明了一种由十个数字组成的数制。然而，数制可以基于任意数量的数字。正如我们已经看到的，双状态电子电路与只含两个数字的数制高度兼容，其数字称为 位 （ 二进制数字 ）。基于八和十六的数制也与计算机等复杂电子逻辑系统兼容，因为它们为表达冗长的二进制数提供了方便的速记方法。

D. 二进制

与几乎所有数字计算机一样， ALTAIR 8800 的几乎所有操作都以二进制执行。计算机处理的典型二进制数包含 8 位，形如：10111010。像这样的定长二进制数通常称为字或字节，计算机通常被设计为处理和存储固定数量的字（或字节）。

像 10111010 这样的二进制字在新手看来毫无意义。但由于二进制只使用两个数字（位），它实际上比我们熟悉而传统的十进制系统简单得多。为了弄清原因，让我们推导十进制数 0 到 20 的二进制等价形式。我们只需简单地给每个后继数加 1，直到推导出所有的数。任何数制中的计数都遵循一条基本规则：在列中顺序记录每次计数的数字；当可用数字全部用完时，在第一列的左侧开始新的一列，并继续计数。

在二进制中从 0 数到 20 非常容易，因为只有两个数字（位）。十进制 0的二进制等价数是 0，十进制 1 的二进制等价数是 1。由于两个可用的位现在都已用完，二进制计数必须引入新的一列，才能构成十进制 2 的二进制等价数，结果是 10。（顺便说一句，不要理会二进制数与十进制数之间任何表面的相似：二进制 10 不是 十进制 10！）十进制数 3 的二进制等价数是 11。两位又都用完了，所以必须开始第三列，才能得到十进制数4 的二进制等价数（100）。现在你应该能够继续计数，推导出十进制数 0到 20 的所有其余二进制等价数：

十进制	二进制
0	0
1	1
2	10

十进制	二进制
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111
16	10000
17	10001

十进制	二进制
18	10010
19	10011
20	10100
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100

十进制	二进制
13	1101
14	1110
15	1111
16	10000
17	10001
18	10010
19	10011
20	10100

有一个简单的过程可以把二进制数转换成它的十进制等价数。二进制数中的每一位指示该数应当取 2 的多少次幂，这些 2 的幂之和就给出该数的十进制等价数。例如，考虑二进制数 10011：

$$\begin{aligned} 10011 &= [(1 \times 2^4) + (0 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times \\ &= [(16) + (0) + (0) + (2) + (1)] \\ &= 19 \end{aligned}$$

E. 八进制

由于二进制只有两位，不多久就会积累出一长串 0 和 1。例如，一个六位十进制数需要 19 位。

冗长的二进制数可以通过把它们分成三位一组、并给每个 3 位组指定十进制等价数来简化。由于最高的 3 位二进制数对应十进制 7，因此有八种 0和 1 的组合（0-7）。

基本的 ALTAIR 8800 接受二进制输入，装入机器的任何二进制数都可以简化为八进制形式。当然，八进制数必须再转换回二进制才能输入计算机，但由于只涉及八种位模式，这个过程既简单又

快速。 ALTAIR 8800 的典型二进制指令是：11101010。该指令可以先从最低有效位开始把数分成三位一组，从而转换为八进制：11 101 010。接下来，给这三个位模式分别指定十进制等价数：

11	101	010
3	5	2

因此，二进制的 11 101 010 对应八进制的 352。为了在本手册其余部分能够快速进行二进制到八进制的转换，大多数二进制数将以三位一组的形式给出。

F. 计算机程序设计

如第 2 部分中将阐明的，计算机的中央处理器（CPU）本质上是一个逻辑电路和系统的网络，其互连或组织可以由用户改变。因此，可以把计算机看作一块 可变硬件 。计算机硬件的变化通过一组称为 软件 的编程指令来实现。

ALTAIR 8800 的软件指令必须以称为 机器语言 的顺序 8 位字的形式装入机器。这种以及其他更高级的计算机语言将在后面讨论。

计算机程序设计的基础相当简单。事实上，程序设计中最困难的部分往往是定义你希望用计算机解决的问题。下面列出生成程序的三个主要步骤：

1. 定义问题2. 确定方法3. 编写程序

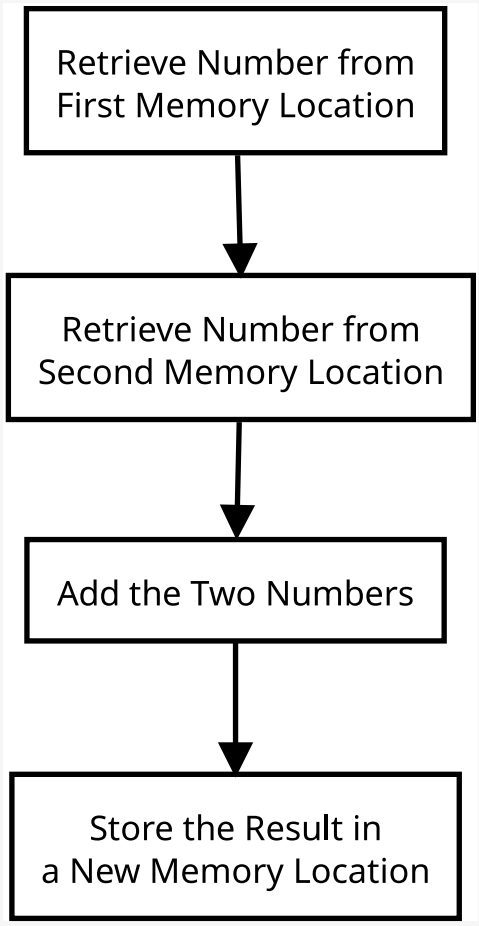
问题一旦得到定义，就可以制定出求解它的方法。绘制一个展示问题有序、逐步求解的图表可以简化这一步，这种图表称为 流程图 。流程图绘好后，各步骤就可以翻译成计算机的语言。这是三个步骤中最容易的一步，因为你只需要对指令有大致的了解，以及一份列出每条指令及其机器语言等价形式的清单。

ALTAIR 8800 具有广泛的程序设计能力。例如，程序可以引起数据在计算机存储器与 CPU 之间传送，甚至可以使计算机做出逻辑判定。例如，如果满足指定条件，计算机可以从程序中一处跳转到任何其他位置，并在新位置继续执行程序。频繁使用的专用程序可以存储在计算机存储器中，供以后由主程序检索和使用，这种专用程序称为子程序。 ALTAIR 8800 的各条指令在本手册第 4 部分中有详细描述。

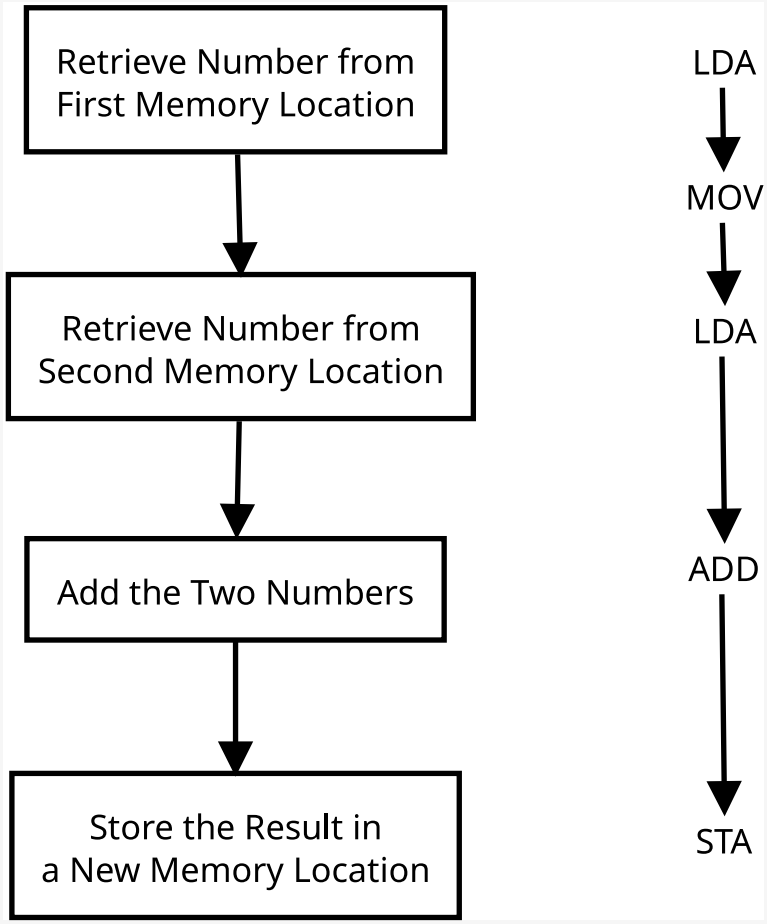
G. 一个简单程序

假设你希望使用 ALTAIR 8800 把位于两个不同存储单元的两个数相加，并把结果存放到存储器的其他地方。当然这是一个很简单的问题，但它可用来说明几种基本程序设计技术。下面是生成解决该问题的程序所用的步骤：

1. 定义问题-把存储器中的两个数相加，并把结果存放到存储器的 其他地方。
2. 确定方法-现在可以生成流程图：



1. 编写程序-起初，把流程图翻译成适合计算机使用的语言或格式 可能显得复杂。然而，对计算机组织和操作的概要了解会使这项工作变 得简单。在这种情况下，四部分流程图翻译成五条独立的指令：



这些指令现在看来可能没有意义，但随着你深入阅读本手册，它们的含义和应用会变得清晰得多。例如，在你了解到计算机必须把从存储器取出的第一个数临时存入一种称为寄存器的特殊 CPU 存储器之后，对额外指令（`MOV`）的需要会变得更加明显。第一个数存放在寄存器中，直到它与第二个数相加。

H. 计算机语言

任何计算机的软件都必须以称为 *机器语言* 的二进制字的形式输入机器。机器语言程序通常借助 *助记符* 编写，这些助记符与各种指令的位模式相对应。例如，`10 000 111` 是 ALTAIR 8800 的一条加法指令，相应的助记符是 `ADD A`。显然，助记符 `ADD A` 比其相应的机器语言位模式容易记得多。

然而最终，每条指令的机器语言位模式都必须一次一步地输入计算机。有些指令可能需要不止一个二进制字。例如，像 `JMP` 这样引用存储地址的 ALTAIR 8800 指令，需要一个字表示实际指令，两个后续的字表示存储地址。

机器语言程序通常通过前面板开关输入 ALTAIR 8800 。计算机终端可用来把助记符信号 `17` 发送给计算机，在那里它由一组称为 *汇编器* 的特殊指令（软件）转换成机器语言。

一种称为 编译器 的高度复杂软件包提供了更大的灵活性，它把高阶助记符转换成机器语言。高阶助记符是一种计算机语言速记方式，用单一且易于识别的助记符自动替换多达十几条甚至更多的机器语言指令。 **FORTRAN** 、 **BASIC** 、 **COBOL** 等高级计算机语言都使用编译器。

更高级的计算机语言在编写计算机程序时提供了大量简化，尤其是那些冗长的程序；它们也非常容易记忆。然而，机器语言程序设计的潜在通用性不应被低估，而实现更高级语言全部潜力的极好方法，就是学习应用机器语言。

第 2 部分：ALTAIR 8800 的组成

图 2-1 展示了 ALTAIR 8800 的组成框图。要有效使用这台计算机，并不 需要详细了解每个部件的电子工作原理；但是，大致了解计算机的各个功能 部分的工作原理是很重要的。

图 2-1：Altair 8800 组成框图

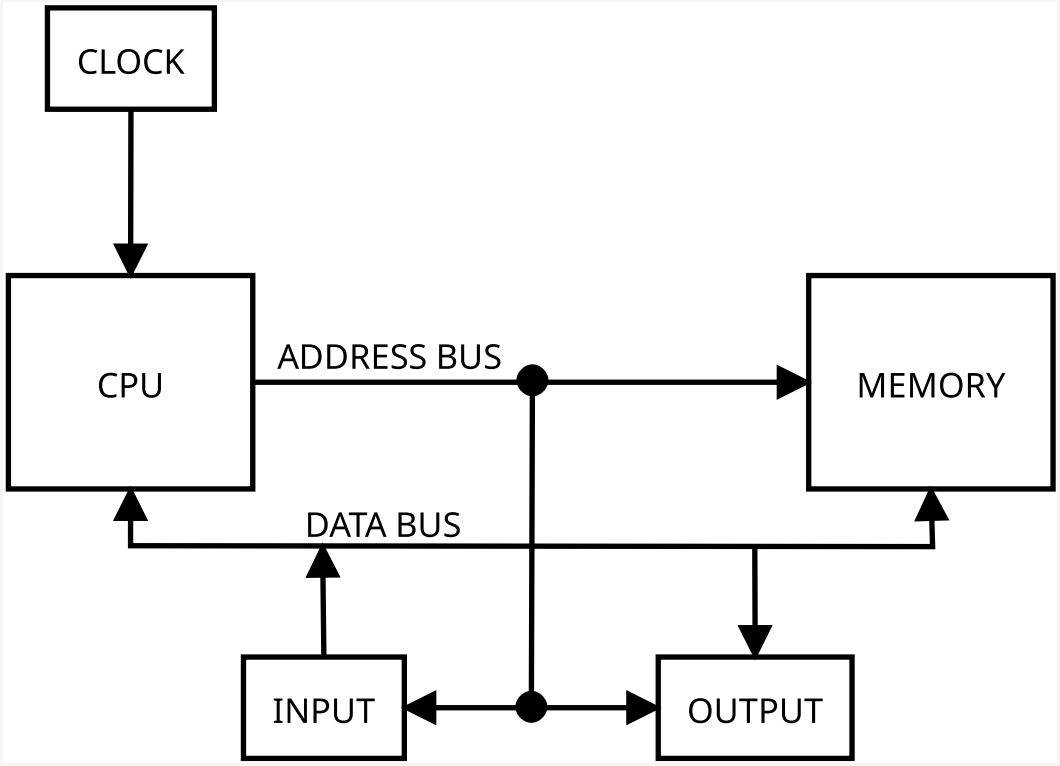
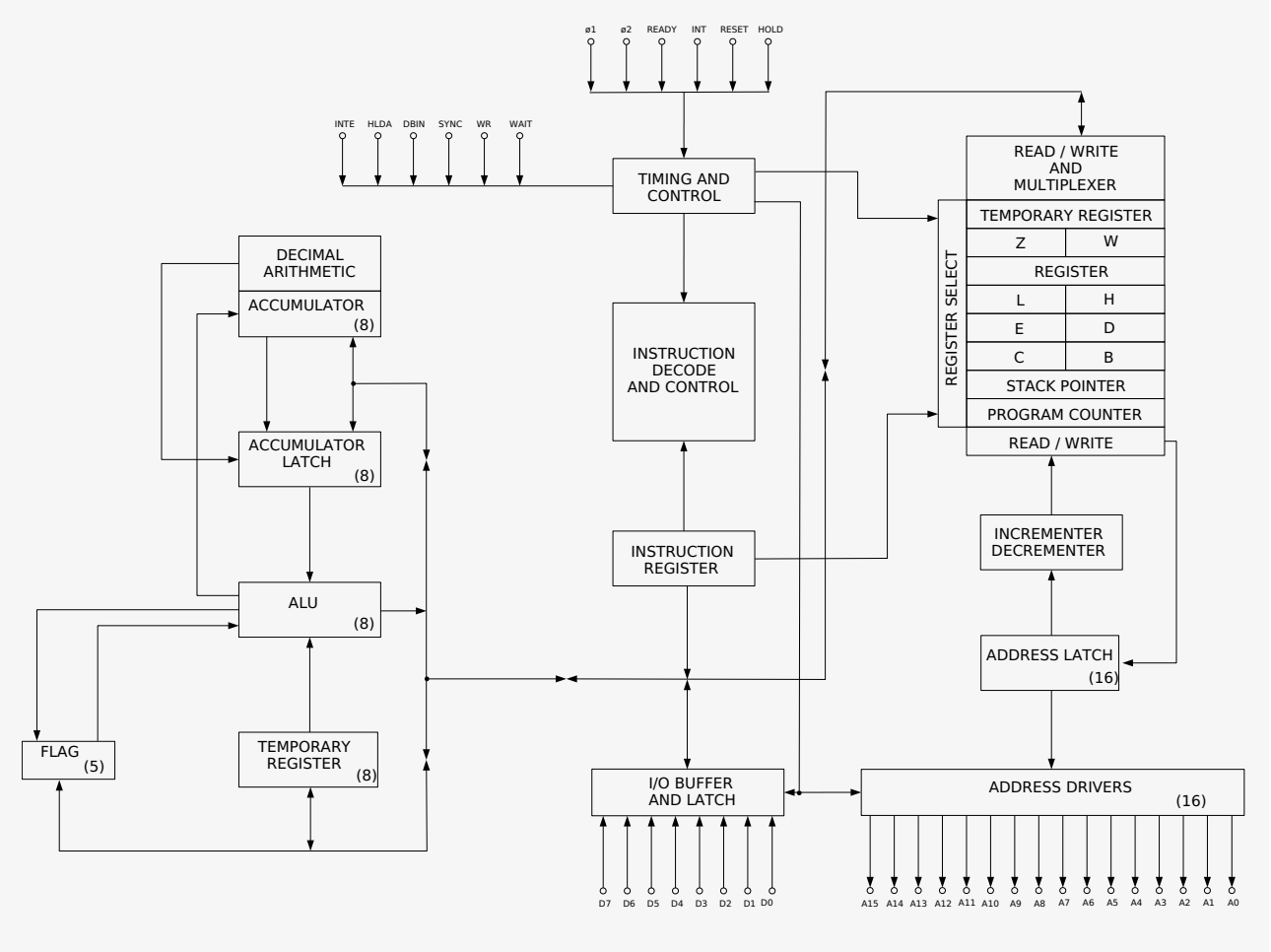


图 2-2：CPU 框图



（点击 CPU 框图查看更大的图片。）

A. 中央处理器（CPU）

中央处理器（CPU）执行所有的算术运算，做出所有的逻辑判断，控制输入 和输出设备对计算机的访问，把数据存入存储器并从存储器中取出数据， 并协调程序的有序执行。CPU 实实在在是计算机的心脏。

当然，必须记住一点：CPU 的智能程度取决于程序员，因为必须用精确的 术语指示 CPU 如何执行某一特定操作。但是，由于 ALTAIR 8800 中的 CPU 执行一个完整的指令周期只需 2 微秒*，计算机能在极短的时间内解 决非常复杂的问题。事实上，ALTAIR 8800 每秒能执行约 30,000 次包含 6 条指令的加法程序。

ALTAIR 8800 小巧紧凑、经济实用，很大程度上要归功于 CPU。得益于 大规模集成电路（LSI）技术，ALTAIR 8800 所使用的 CPU 被制造在一 小块硅芯片上，其表面积只有几分之一英寸。这款芯片就是 Intel 8080，它安装在带有 40 个引脚的保护性双列直插式封装中。

CPU 显然是 ALTAIR 8800 中最复杂的部分。图 2-2 给出了 CPU 的完整 框图。虽然不需要详细理解这张图，但了解 CPU 中一些较重要子系统的 功用是很重要的。这样一来，这些子系统之间的相互关系以及它们对 CPU 运行的贡献就会变得更加清楚。

1. 时序与控制-时序与控制系统从时钟接收时序信号，并将其分配 给 CPU 的相应部分，以确保指令执行的协调一致。时序与控制系统还 会激活前面板上的若干状态指示灯（HOLD、WAIT、INTE、STACK、OUT、 IN、INP、MI MENR、HLTA、WO、INT）。

   ◦ 微秒是百万分之一秒。

2. 指令寄存器-二进制机器语言指令暂时存放在指令寄存器中，供 CPU 译码和执行。

3. 算术运算-算术运算系统既能执行二进制算术运算，也能执行十 进制算术运算。所有算术运算都通过加法完成。乘法通过重复加法实 现，减法和除法通过逆加法实现。

4. 工作寄存器-CPU 包含 7 个 8 位工作寄存器。其中最重要的是 累加器，许多运算的结果最终都会装入累加器。除了作为许多程序运算 结果的主要存放点之外，累加器还可以与任一指定的寄存器或存储地址 进行大量的算术和逻辑运算，

   其余 6 个寄存器成对排列，以便在需要时进行 16 位运算，它们是 “暂存”寄存器。这仅仅意味着它们通常用于存放临时数据或地址，并可 用于大量的程序运算。

   图 2-3 展示了 7 个工作寄存器的排列和分类。紧邻累加器的另一个寄 存器是状态位寄存器，它是一个专用寄存器，用于存放某些运算的状 态。

图 2-3. 工作寄存器

寄存器对 B	→	B	C
寄存器对 D	→	D	E
寄存器对 H	→	H	L
寄存器对 PSW	→	*	A

* 状态位寄存器（见正文）

1. 状态位寄存器-状态位寄存器是一个专用寄存器，用于存放 5 种 状态，这些状态可能受数据运算结果的影响，也可能不受影响。该寄存 器包含 8 个位的位置，但只有 5 位用于存放状态信息。这 5 个状态位 是：

   1. 进位位-如果发生进位，该位置 1。加法、减法、循环移位以 及某些逻辑判断等运算通常会影响进位位。如果没有发生进位，该位 置 0。

2. 辅助进位位-该位置 1 表示结果的第 3 位有进位。0 表示无 进位。该状态位只受一条指令（DAA）影响。

3. 符号位-该位置位用于表示结果的正负。置 1 表示结果为 负；置 0 表示结果为正。符号位反映结果中最高有效位（第 7 位） 的状态。这是因为，如果用最高有效位指示结果的正负，一个 8 位字 节能表示的十进制数范围为 -128 到 +127。23

4. 零位-如果某些指令的结果为零，该位置 1；如果结果大于 零，则该位复位为 0。

5. 奇偶位-某些运算会检查结果的奇偶性。奇偶性表示结果中 1 位的个数是奇数还是偶数。因此，如果 1 的个数为偶数，奇偶位置 1；如果 1 的个数为奇数，奇偶位置 0。

2. 程序计数器-程序计数器是一个特殊的 16 位寄存器，用于存放 下一条待执行程序步骤的地址。每执行完一个步骤，程序计数器就自动 前进到下一个顺序程序地址。程序计数器有时也称为 P 计数器，程序员 可以通过实现 JUMP、CALL 和 RETURN 指令的机器语言指令直接访问 它。

3. 堆栈指针-堆栈指针是另一个特殊的 16 位寄存器。存储器中保 留用于临时存放数据或地址的区域称为堆栈。

   可以通过若干条指令把数据压入堆栈临时存放，也可以从堆栈中弹出数 据。

   堆栈指针用于在执行子程序期间保存程序计数器的内容。RETURN 指令把 堆栈指针的内容传送给程序计数器，主程序随即继续顺序执行。程序员 通过一条专用指令（LXI）把期望的存储地址装入堆栈指针，从而选择 堆栈在存储器中的位置。

工作寄存器、程序计数器、堆栈指针、算术运算系统、指令寄存器以及时 序与控制系统之间的相互关系，现在应该更有意义了。工作寄存器包括 6 个暂存寄存器和 1 个累加器，大量运算结果会临时存放在累加器中。程序 计数器通过跟踪下一条待执行指令的存储地址，使程序得以顺序执行。时序与控制系统提供时序脉冲，协调程序的有序执行。堆栈指针用于临时存 放任一寄存器对中的数据。堆栈指针还会在子程序执行前保存程序计数器 中的地址，以便子程序执行完毕后取回。所有这些操作合在一起，构成了 一个极其灵活、用途广泛的 CPU。

B. 存储器

尽管工作寄存器、程序计数器和堆栈指针确实起着存储器的作用，但 CPU 并不包含计算机应用中所定义的那种存储器。计算机的主存储器位于 CPU 外部。

简单的程序用几十个字甚至更少的存储器就能实现，但像视频处理这样更 复杂的应用则需要更多存储器。ALTAIR 8800 的存储器最多可扩展到 65,536 个 8 位字。

存储器的访问始终由 CPU 控制。* 16 条称为地址总线的线路把 CPU 与存 储器连接起来。这些线路使 CPU 能够向任一存储地址输出数据或从任一存 储地址输入数据。地址由两个 8 位字节指定。CPU 把每个地址当作两个顺 序（串行）周期来处理，每个周期包含 8 个并行位。存放在存储器中

的数 据通过 8 条称为数据总线的数据线在存储器与 CPU 之间交换。这种互连 格式支持并行操作。因此，当 CPU 从存储器读取数据或向存储器写入数据 时，数据以完整的 8 位字形式传送。

ALTAIR 8800 的基本存储器最多包含 8 个 256 x 4 位随机存取存储器 （RAM）。不过，只要总线上的输入负载不超过 50 个 TTL 负载，并且总 线由标准 TTL 负载驱动，计算机中可以使用任何常规存储器。

- 例外情况是计算机连接到了直接存储器访问控制器。DMA 在直接传送整 块数据时从 CPU 接管地址线和数据线。这些传送可以在内部进行（从一 个存储单元传送到另一个存储单元），也可以在外部进行（从存储器传送 到外部设备）。

C. 时钟

CPU 对程序的有序执行由 2 MHz 晶体控制时钟控制。采用晶体控制是为了 让时钟以 CPU 所允许的最高速度运行。没有晶体调节的时钟有时可能会 加速到超出 CPU 的能力，从而导致程序执行错误。

D. 输入/输出

ALTAIR 8800 可以与大量外部设备接口。一般来说，这些设备向计算机 提供输入信息，并接收来自计算机的输出信息。CPU 监视程序执行以及输 入/输出设备的状态，并为外部设备提供服务提供必要的信号。程序员可以 指示 CPU 忽略或响应外部设备提供的中断信号。这些中断信号一旦被 CPU 接受，就会使程序执行暂时停止，同时由计算机为该外部设备提供服务。 外部设备服务完毕后，程序恢复正常执行。ALTAIR 8800 最多可为 256 个输入设备和 256 个输出设备提供服务。

至此，ALTAIR 8800 的组成已介绍完毕。在第 3 部分关于 ALTAIR 8800 操作的讨论中，这台计算机作为一台功能强大、效率很高的数据处 理系统的整体运行情况将会变得更加清楚。

第 3 部分：ALTAIR 8800 的操作

基本型 ALTAIR 8800 通过前面板进行操作。乍一看，25 个拨动开关和 36 个指示及状态 LED 的阵列可能令人困惑。实际上，ALTAIR 8800 的操作非 常直接，大多数用户不到一小时就能学会把程序装入机器并运行它。如果你 是典型用户，你花在开发和编写程序上的时间将远远超过实际操作机器的时 间。

ALTAIR 8800 操作手册的这一部分介绍前面板开关以及指示和状态 LED 的 用途和应用。接着把一个示例程序装入机器并运行。随后详细讨论存储器的 作用和有效使用。最后给出几条操作提示，帮助你编辑和“调试”程序。

A. 前面板开关与 LED

尽管前面板包含 25 个拨动开关和 36 个指示及状态 LED，但基本型 ALTAIR 8800 （256 字存储器）的大多数常规操作只需使用 15 个开关并监 视 16 个 LED 即可完成。下面说明所有开关和 LED 的功能：

1. 前面板开关

ON-OFF 开关

ON 位置为计算机接通电源。**OFF** 位置切断电源，同时也会清除 存储器的内容。

STOP-RUN 开关

STOP 位置停止程序执行。**RUN** 位置执行程序。

SINGLE STEP 开关

每拨动一次该开关，就执行一条机器语言指令。一条机器语言指令最多 可能需要 5 个机器周期。

EXAMINE-EXAMINE NEXT 开关

EXAMINE 位置在 8 个数据 LED 上显示先前通过 **DATA/ADDRESS** 开关（见下文）装入的任何指定存储地址的内容。**EXAMINE NEXT** 位 置显示下一个顺序存储地址的内容。每拨动一次 **EXAMINE NEXT**，都 会显示下一个顺序存储地址的内容。

DEPOSIT-DEPOSIT NEXT 开关

DEPOSIT 位置把装入 8 个 **DATA** 开关的数据字节装入先前指定 的存储地址。**DEPOSIT NEXT** 位置把装入 8 个 **DATA** 开关的数据 字节装入下一个顺序存储地址。每拨动一次 **DEPOSIT NEXT**，装入 8 个 **DATA** 开关的数据字节就会被装入下一个顺序存储地址。在

拨动 **DEPOSIT** 或 **DEPOSIT NEXT** 之前，可以更改装入 8 个 **DATA** 开关的数据字节。

RESET-CLR 开关

RESET 位置把程序计数器设置为第一个存储地址（0 000 000 000 000 000）。对于从第一个存储地址开始的程序，**RESET** 提供了一种 快速有效的方法回到程序的第一步。**CLR** 是用于外部输入/输出设备 的 **CLEAR** 命令。

PROTECT-UNPROTECT 开关

PROTECT 位置防止存储器内容被更改。**UNPROTECT** 位置允许更改 存储器的内容。

AUX 开关

基本型 ALTAIR 8800 带有两个辅助开关，它们尚未连接到计算机。这 些开关将与添加到基本机器上的外部设备配合使用。

DATA/ADDRESS 开关

DATA 开关是指定为 7-0 的那些开关。**ADDRESS** 开关是指定为 15-0 的那些开关。拨动开关处于 **UP** 位置表示 1 位。拨动开关处于 **DOWN** 位置表示 0 位。在基本型 ALTAIR 8800 （256 字存储器） 中，指定为 8-15 的 **ADDRESS** 开关不使用，输入地址时应将其置 0。

2. 指示 LED

> **备注**
>
> 机器停止时，点亮的 LED 表示 1 位或指定条件的活动状态；未点亮的 LED 表示 0 位或非活动状态。但是，在运行程序时，LED 的显示可能 看起来不正确。

LED	定义
ADDRESS	**ADDRESS** LED 是指定为 **A15**-**A0** 的那些。**ADDRESS** LED 上显示的位模式表示所检查 或装入数据的存储地址。
DATA	**DATA** LED 是指定为 **D7**-**DO** 的那些。**DATA** LED 上显示的位模式表示指定存储地 址中的数据。
INTE	该 LED 点亮时，表示中断（**INTERRUPT**）已允许。

LED	定义
PROT	该 LED 点亮时，表示存储器已处于保护（**PROTECTED**）状态。
WAIT	该 LED 点亮时，表示 CPU 处于等待（**WAIT**）状态。
HLDA	该 LED 点亮时，表示 **HOLD** 已获确认（**ACKNOWLEDGED**）。

3. 状态 LED

> **备注**
>
> 点亮的 LED 表示指定条件处于活动状态。

LED	定义
MEMR	存储器总线将用于存储器读数据。
INP	地址总线上含有输入设备的地址。当数据总线处于输入模 式时，输入数据应放到数据总线上。
M1	CPU 正在处理一条指令的第一个机器周期。
OUT	地址总线上含有输出设备的地址；当 CPU 就绪时，数据总线上将包 含输出数据。
HLTA	一条 **HALT** 指令已执行并得到确认。
STACK	地址总线上保存着堆栈指针的下压堆栈地址。
WO	

LED	定义
	当前机器周期将执行 **WRITE** 存储器或 **OUTPUT** 功能。否 则，将执行 **READ** 存储器或 **INPUT** 操作。
INT	一个 **INTERRUPT** 请求已被确认。

B. 载入示例程序

第 1 部分 G 节介绍了一个用机器语言助记符编写的简单加法程序。该程序 旨在从存储器中取出两个数，把它们相加，并把结果存入存储器。用助记符 形式表示，该程序可以这样写：

1. `LDA`
2. `MOV` (A→B)
3. `LDA`
4. `ADD` (A+B)
5. `STA`
6. `JMP`

ALTAIR 8800 全部 78 条指令的助记符在本手册第 4 部分中有详细说 明。目前，以下定义已经足够：

1. `LDA` –把指定存储地址的内容装入累加器。
2. `MOV` (A→B)–把累加器的内容传送到寄存器 B。
3. `ADD` (B+A)–把寄存器 B 的内容加到累加器的内容上，并 把结果存入累加器。
4. `STA` –把累加器的内容存入指定的存储地址。
5. `JMP` –跳转到程序的第一步。

备注

计算机执行完程序后，会在存储器中寻找其他可做的事情。为了保持对 CPU 的控制，我们可以用一条 `JMP` 指令（后跟第一条指令的存储地 址）来结束示例程序。计算机将“跳转”回示例程序的第一条指令，一遍 又一遍地执行该程序。

注意这些指令每一条是多么精确和具体。计算机被 确切地 告知如何解 决问题以及把结果放到哪里。这些机器语言指令每条都需要一个单字节位 模式来实现基本指令。`LDA` 和 `STA` 还需要两个额外字节来提供必 要的存储地址。

要把这个程序装入 ALTAIR 8800，必须先确定两个待相加的数、结果以 及程序本身的存储地址。在大多数情况下，从第一个存储地址（0）开始存 放新程序会更方便。因此，数据（两个待相加的数和结果）的存储地址应放 在存储器中较高的任意地址上。由于基本型 ALTAIR 8800 有 256 字存储 器，我们选择从存储地址 128 开始存放数据。第一个加数放在存储地址 128（10 000 000），第二个加数放在存储地址 129（10 000 001），结果放 在存储地址 130（10 000 010）。存储地址确定后，就可以把程序转换成它 的机器语言位模式了：

助记符	位模式	说明
LDA	00 111 010 10 000 000 00 000 000	把以下位置的内容装入累加器： 存储地址 128 （存储地址需要 2 个字节）
MOV (A→B)	01 000 111	把累加器的内容传送到寄存器 B
LDA	00 111 010 10 000 001 00 000 000	把以下位置的内容装入累加器： 存储地址 129
ADD (B+A)	10 000 000	把寄存器 B 的内容加到累加器
STA	00 110 010 10 000 010 00 000 000	把累加器的内容存入： 存储地址 130
JMP	11 000 011 00 000 000 00 000 000	跳转到： 存储单元 0

通常，机器语言程序的各个位模式会按顺序编号，以减少把它们输入计算机 时出错的可能性。此外，由于用前面板开关装入八进制数非常容易，通常还 会给出每个位模式的八进制等价值。所需要做的只是记住十进制数 0-7 的二 进制/八进制等价值。得到的程序可以这样表示：

步骤	助记符	位模式	八进制等价值
0	LDA	00 111 010	072
1	（地址）	10 000 000	200
2	（地址）	00 000 000	000
3	MOV (A→B)	01 000 111	107
4	LDA	00 111 010	072
5	（地址）	10 000 001	201
6	（地址）	00 000 000	000
7	ADD (B+A)	10 000 000	200
8	STA	00 110 010	062
9	（地址）	10 000 010	202
10	（地址）	00 000 000	000
11	JMP	11 000 011	303
12	（地址）	00 000 000	000

步骤	助记符	位模式	八进制等价值
13	（地址）	00 000 000	000

现在可以用前面板开关把程序输入计算机了。要从第一个存储地址（0）开始装入程序，请拨动 **RESET** 开关。程序计数器随即装入第一个存储地址。然后一步一步地把程序输入 **DATA/ADDRESS** 开关 7-0。第一步输入后，拨动 **DEPOSIT** 开关，把位模式装入存储器。接着把第二步输入 **DATA/ADDRESS** 开关，并拨动 **DEPOSIT NEXT** 开关。位模式将自动装入下一个顺序存储地址（1）。继续把各步骤输入前面板开关并拨动 **DEPOSIT NEXT**。完整的程序装入过程可以总结如下：

步骤	开关 0-7	控制开关
		RESET
0	00 111 010	
		DEPOSIT
1	10 000 000	
		DEPOSIT NEXT
2	00 000 000	
		DEPOSIT NEXT
3	01 000 111	
		DEPOSIT NEXT
4	00 111 010	

步骤	开关 0-7	控制开关
		DEPOSIT NEXT
5	10 000 001	
		DEPOSIT NEXT
6	00 000 000	
		DEPOSIT NEXT
7	10 000 000	
		DEPOSIT NEXT
8	00 110 010	
		DEPOSIT NEXT
9	10 000 010	
		DEPOSIT NEXT
10	00 000 000	
		DEPOSIT NEXT
11	11 000 011	
		DEPOSIT NEXT

步骤	开关 0-7	控制开关
12	00 000 000	
		DEPOSIT NEXT
13	00 000 000	
		DEPOSIT NEXT

程序现在可以运行了，但首先必须在两个待相加的存储地址中存入数据。要 装入第一个地址，把 **DATA/ADDRESS** 开关设置为 10 000 000 并拨动 **EXAMINE**。现在可以通过相应设置 **DATA/ADDRESS** 开关，把任何期 望的数值装入该地址。数值装入开关后，拨动 **DEPOSIT** 把它装入存储 器。要装入下一个地址，在 **DATA/ADDRESS** 开关上输入第二个数并拨动 **DEPOSIT NEXT**。由于选择的是顺序存储地址，该数将自动装入正确的地 址（10 000 001）。如果选择的是非顺序存储地址，则必须遵循查找第一个 地址的过程（把地址装入 **DATA/ADDRESS** 开关并拨动 **EXAMINE**；然 后把数值装入 **DATA/ADDRESS** 开关并拨动 DEPOSIT）。

现在程序引用的两个存储地址已经装入了两个待相加的数，可以运行程序 了。这只需拨动 **RESET** 开关，然后再拨动 **RUN** 开关即可。稍等片 刻，然后拨动 **STOP** 开关。要查看存入存储器的结果，先用 **DATA/ADDRESS** 开关设置结果所在地址的位模式（10 000 010），然后 拨动 **EXAMINE** 开关。结果就会显示在 **DATA** LED 上。

要检验自己装入和运行这个程序的能力，请尝试更改两个加数和结果的存储 地址，然后再次装入并运行该程序。

二进制乘法示例程序

地址	助记符	八进制 代码	说明
000 001	MVIA	076 002	乘数送入 A 寄存器
002 003	MVID	026 003	被乘数送入 D、E 寄存器
004 005	MVIE	036 000	

地址	助记符	八进制 代码	说明
006 007 010	LXIH	041 000 000	清零 H、L 寄存器以初始化部分积
011 012	MVIB	006 010	迭代次数送入 B 寄存器
013	DADH	051	部分积连同进位左移
014	RAL	027	乘数位移入进位位
015 016 017	JNC	322 023 000	在进位位处测试乘数
020	DADD	031	若进位位 =1，把被乘数 加到部分积上
021 022	ACI	316 000	
023	DCRB	005	递减迭代计数器
024 025 026	JNZ	302 013 000	检查迭代次数
027 030 031	SHLD	042 100 000	把结果存入存储单元 100、101
032 033 034	JMP	303 000 000	重新启动

C. 存储器的使用

到现在应该很清楚，存储器在计算机的高效运行中起着至关重要的作 用。高级语言编译器通常带有一个软件包，自动跟踪各个存储地址。然 而，机器语言操作要求程序员自己跟踪存储器。否则，宝贵的数据或程序 指令可能会被意外擦除，或被其他数据或指令替换。

可以通过一种称为 存储器映射 的简单技术来跟踪 ALTAIR 8800 存储 器中存放的内容。这种技术只是把各种类型的数据分配给为特定目的保留 的特定存储块。该技术能有效地把可用存储器组织成一个高效且易于访问 的存储介质。

对于拥有 256 字存储器的 ALTAIR 8800，典型的存储器映射可能把前 100 字分配给程序，第二个 100 字分配给子程序，剩下的 56 字分配给数 据。当然，各个存储块可以随意修改，存储器映射的主要目的是对可用存 储器进行有组织的统一规划。

每次更改 ALTAIR 8800 中的程序时，都可以制作一份存储器映射。程序 写好后，决定应为程序本身、子程序（如果有）和数据保留多少存储空 间。然后制作一个表格或图表，记录各项内容存放在存储器的什么位置。 存储器组织更改时，务必更新该表格。

D. 存储器寻址

ALTAIR 8800 的机器语言指令集提供了多种存储器寻址方式，包括直接 寻址、寄存器对寻址、堆栈指针寻址、立即寻址以及子程序的堆栈寻 址。下面介绍这几种寻址方式。

1. 直接寻址-指令以两个字节的形式提供指定的存储地址，这 两个字节紧跟在实际指令字节之后。

2. 寄存器对寻址-寄存器对的内容可以存放存储地址。在大多 数指令中，必须为此使用 `H` 和 `L` 寄存器。`H` 寄存器存放最 高有效 8 位，`L` 寄存器存放最低有效 8 位（`H` 为 高， `L` 为低）。两条指令（`STAX` 和 `LDAX`）允许 `B` 和 `C` 或 `D` 和 `E` 寄存器对存放存储地址。

3. 堆栈指针寻址-堆栈操作只有两种：`PUSH` 和 `POP`。把数据 PUSH 到堆栈上，会把两个字节（16 位）数据存入程 序员保留的、称为堆栈的特定存储块。从堆栈中 POP 数据则会取出这些 数据。`PUSH` 和 `POP` 指令在本手册 第 4 部分 中有详细说 明。目前必须知道的是，程序员 必须 通过把存储地址装入堆栈指针 来保留存储器中的堆栈位置。这通过 `LXI` 指令实现（见 第 4 部分）。程序员应始终在自己的存储器映射上记下堆栈的地 址。

4. 立即寻址-立即寻址指令包含在程序装入期间装入存储器的 数据。由于数据与程序一起按顺序装入，它存放在操作者为程序保留的 存储块中。装入立即数时无需更改存储器映射。

5. 子程序的堆栈寻址-当程序调用（CALL）一个子程序时，主 程序中下一条顺序指令的地址会自动地通过压入（PUSH）堆栈保存。子 程序执行完毕后，RETURN 指令从堆栈弹出（POP）该地址，主程序继续 执行。

E. 操作提示

随着你对 ALTAIR 8800 操作经验的积累，你会想出一些方法来提高程序 的效率和计算机的运行水平。下面列出了几条有用的提示，在学习操作这 台机器时你会发现它们非常有用。

1. 校对程序-为了安全起见，程序输入计算机后一定要进行校 对。方法是回到程序开始的第一个存储地址（如果程序从存储单元 0 开 始，拨动 **RESET**；否则在 **ADDRESS** 开关上设置地址并拨动 **EXAMINE**），检查 **DATA** LED，确认第一步程序已正确输入。然 后拨动 **EXAMINE NEXT**，对照 **DATA** LED 检查第二步。以这种方 式继续校对，直到检查完整个程序。如果发现错误，只需在 **DATA** 开 关上重新输入正确的位模式，拨动 **DEPOSIT**，然后继续用 **EXAMINE NEXT** 开关进行校对。

2. 使用 **NOP**-`NOP` 是一条指定“无操作”的指令，看起来没 什么价值。但是，如果在复杂程序中散布 `NOP` 指令，那么当发现需 要添加新步骤的程序错误时，可以节省大量时间。在程序校对期间，只 需把新指令或数据输入到 `NOP` 指令的位置上即可。如果觉得某条新 指令可能是必要的，务必使用适当数量的 `NOP`。例如，如果你认为 程序执行不正常时可能需要添加一条 `LDA` 指令，就在需要的位置连 续使用三条 `NOP`。需要三条 `NOP` 是因为 `LDA` 指令需要三 个独立的字节。

3. 调试程序-有时需要对程序进行“调试”。当程序因错误 （bug）而无法正常执行时，就需要调试。使用 **SINGLE STEP** 开关可 以帮助调试。该开关让计算机按机器周期（而不是完整的程序步骤）步 进执行程序，使你能观察 8 个 **STATUS** LED 的状态。这个过程能让 你发现非法输入、程序组织不当以及其他编程错误。

第 4 部分：Altair 8800 指令集

ALTAIR 8800 有 78 条基本机器语言指令。由于其中许多指令可以经过修改以作 用于不同的寄存器或寄存器对，因此这些基本指令共有 200 多种变化形式。

本操作手册的其余部分将详细介绍 ALTAIR 8800 的指令集。为此，这 78 条基 本机器语言指令被划分为七个主要类别：

1. 命令指令
2. 单寄存器指令
3. 寄存器对指令
4. 累加器循环移位指令
5. 数据传送指令
6. 立即数指令
7. 转移指令

每条指令都以标准化的助记符或机器语言代码的形式给出。指令可能占用一到三个 顺序（串行）字节，并给出了相应的位模式。完整指令集的精简汇总（以二进制和 八进制列出全部助记符和指令）见附录。

A. 命令指令

ALTAIR 8800 有九条专用命令指令，用于为其余指令提供服务。这些专用指令分 为四类：输入/输出指令（`IN`、`OUT`）、中断指令（`EI`、`DI`、`HL T`、`RST`）、进位位指令（`STC`、`CMC`）和空操作指令（`NOP`）。

1. 输入/输出指令

有两条输入/输出指令，每条占用两个字节。第一个字节是指令，第二个字节是输 入/输出设备号。

助记符	指令	二进制代码	位置
`IN`	INPUT	11 011 011	

助记符	指令	二进制代码	位置
			(Byte 1)
	（设备号）		(Byte 2)

操作：将 8 位数据字节从指定的外部设备装入累加器。

状态位：不受影响。

示例：假设某个输入设备包含如下数据字节：`00 001 000`。执行 `IN `` 指令（包括 ``设备号`）将使该数据字节取代累加器的内容。

助记符	指令	二进制代码	位置
`OUT`	OUTPUT	`11 010 011`	(Byte 1)
	（设备号）		(Byte 2)

操作：将 8 位数据字节从累加器装入指定的输出设备。

状态位：不受影响。

示例：假设累加器包含如下数据字节：`00 001 000`。执行 `OUT` 指令（外加 `设备号`）将使该数据字节被发送到指定的外部设备。

2. 中断指令

有两条特定的中断指令（`EI` 和 `DI`）以及两条辅助中断指令。中断指令使 计算机执行的程序可以暂时被打断，以便进行输入/输出接口操作。例如，当某个 输入设备正在输入数据或程序时，计算机的输出设备可以利用中断。

助记符	指令	二进制代码	位置
`EI`	ENABLE INTERRUPTS	`11 111 011`	(Byte 1)
操作：执行 `EI` 指令将置位中断触发器。这使计算机注意到中断 的到来并作出相应响应。			
状态位：不受影响。			

助记符	指令	二进制代码	位置
`DI`	DISABLE INTERRUPTS	`11 110 011`	(Byte 1)
操作：执行 `DI` 指令将复位中断触发器。这使计算机忽略其后到 来的任何中断信号。			
状态位：不受影响。			

助记符	指令	二进制代码	位置
`HLT`	HALT INSTRUCTION	`01 110 110`	(Byte 1)
操作：执行 `HLT` 指令将把程序计数器步进到下一条指令的地址， 并使计算机停机，直到发生中断。在执行过 `DI` 指令之后，通常不应 再执行 `HLT` 指令。由于 `DI` 指令使计算机忽略中断，因此在关闭 主电源开关再重新打开之前，计算机不会再运行。			

助记符	指令	二进制代码	位置
状态位：不受影响。			

助记符	指令	二进制代码	位置
`RST`	RESTART INSTRUCTION	11 **(exp)** 111	(Byte 1)
操作：程序计数器中的数据字节被压入堆栈。这为随后执行 `RETURN ``` 指令提供了一个地址。程序随后从存储地址 ```00 000 000 00 (exp) 0 00` 处继续执行，其中 `exp` 的取值范围为 `000` 到 `111`。  `RST` 指令通常用于中断服务。在中断期间，外部设备可以使 `RST` 指令被执行。`RST` 的执行会调用一个专用子程序，该子程序存储在存 储器最低 64 个字中、最多 8 个 8 位字节的范围内。子程序中包含一条 `RETURN` 指令，使计算机返回原来的程序。			
状态位：不受影响。			
示例：假设存在如下 `RST` 指令：`11 001 111`。该指令的执行 将使程序计数器的数据字节被压入堆栈。程序随后将从位于存储地址 ```00 000 000 00 001 000`` 的子程序处继续执行。子程序执行完毕后，一条 `RETURN` 指令将使计算机返回主程序中的下一步。			

3. 进位位指令

有两条指令可用于直接修改进位位的状态。每条指令需要一个 8 位字节。

助记符	指令	二进制代码	位置
`CMC`	COMPLEMENT CARRY	00 111 111	(Byte 1)

助记符	指令	二进制代码	位置

操作：进位位被取反。如果它初始为 0，则置为 1；如果初始为 1， 则复位为 0。

受影响的状态位：进位位。

助记符	指令	二进制代码	位置
STC	SET CARRY	00 110 111	(Byte 1)

操作：进位位被置为 1。

受影响的状态位：进位位。

4. 空操作指令

有一条空操作指令。它占用单个 8 位字节。

助记符	指令	二进制代码	位置
NOP	NO OPERATION	00 000 000	(Byte 1)

操作：不发生任何操作，程序计数器顺序推进到下一条指令，程序随 后继续执行。

状态位：不受影响。

B. 单寄存器指令

ALTAIR 8800 有四条单寄存器指令。每条指令占用一个字节。其中两条指令 `I NR` 和 `DCR` 各有八种变化形式。这些变化形式通过指定任一所需的寄存器来 确定，适用的寄存器位模式如下：

寄存器	位模式
B	000
C	001
D	010
E	011
H	100
L	101
Memory Reference M	110
A	111

如果在指令字节中指定 `Memory Reference M``(110),则处理由 ``H` 和 `` L`` 寄存器的内容寻址的存储字节。`H` 寄存器包含存储地址的最高有效 8 位， `L` 寄存器包含该地址的最低有效 8 位。

助记符	指令	二进制代码	位置
`INR`	INCREMENT REGISTER OR MEMORY	00 **(reg)** 100	(Byte 1)
操作：指定的字节加 1。			

助记符	指令	二进制代码	位置

受影响的状态位：零位、符号位、奇偶位和辅助进位位。

示例：假设存在如下指令：`00 000 100`。根据上文给出的寄存器 位模式表，寄存器 `B` 中的字节将加 `1`。如果初始字节为 `00 000 000`， 则加 `1` 后的字节为 `00 000 001`。

助记符	指令	二进制代码	位置
`DCR`	DECREMENT REGISTER OR MEMORY	`00` **(reg)** `101`	(Byte 1)

操作：指定的字节减 `1`。

受影响的状态位：零位、符号位、奇偶位和辅助进位位。

示例：假设存在如下指令：`00 001 101`。根据上文给出的寄存器 位模式表，寄存器 `C` 中的字节将减 `1`。如果初始字节为 `00 000 001`， 则减 `1` 后的字节为 `00 000 000`。

助记符	指令	二进制代码	位置
`CMA`	COMPLEMENT ACCUMULATOR	`00 101 111`	(Byte 1)

操作：累加器中的每一位被取反（`1` 变为 `0`，`0` 变为 `1`）。

状态位：不受影响。

示例：假设累加器字节为 `11 001 100`。`CMA` 指令将按如下方式取 反累加器字节中的每一位：

```
       11 001 100          累加器
```

助记符	指令	二进制代码	位置
	————		
	00 110 011 取反后的 累加器		

助记符	指令	二进制代码	位置
DAA	DECIMAL ADJUST ACCUMULATOR	00 100 111	(Byte 1)

操作：8 位累加器字节被转换为两个 4 位 BCD（二进制编码的十进制）数。该指令受辅助进位位影响。DAA 指令执行两项操作：

1. 如果累加器字节中最低有效的 4 位（位 0-3）表示的 **BCD** 数字大于 9，或者辅助进位位被置为 1，则这四位自动加 6。否则，累加器 不受影响。
2. 如果累加器字节中最高有效的 4 位（位 4-7）表示的 **BCD** 数字大于 9，或者上一步操作之后进位位被置为 1，则这四位自动加 6。否 则，累加器不受影响。

受影响的状态位：零位、符号位、奇偶位、进位位和辅助进位位。

示例：假设累加器字节为 10 100 100。DAA 指令将自动把该字节 视为两个 4 位字节：1010 0100。由于最低有效的 4 位的值小于 9，累加 器起初不受影响。而最高有效的 4 位的值大于 9，因此这四位加 6 得到 1 0000。最高有效位将进位位置为 1，累加器现在的内容为：00 000 100。

C. 寄存器对指令

ALTAIR 8800 有八条寄存器对指令。

每条指令占用一个字节。其中五条指令 PUSH、POP、DAD、INX 和 DCX 各有四种变化形式。这些变化形式通过指定任一所需的寄存器对来确 定，适用的寄存器对位模式如下：

寄存器对	位模式
B and C	00
D and E	01
H and L	10
Flags and A	11

助记符	指令	二进制代码	位置
PUSH	PUSH DATA ONTO STACK	11 **(rp)**0 101	(Byte 1)

操作：指定寄存器对 `(rp)` 的内容被存储到由堆栈指针指示的地 址处的两个存储字节中。第一个寄存器的内容被 `PUSH` 到比堆栈指针 中的地址小 1 的地址，第二个寄存器的内容被 `PUSH` 到比堆栈指针中 的地址小 2 的地址。如果指定的是由状态位寄存器和累加器组成的寄存器 对（PSW），则压入存储器的第一个字节是状态位寄存器。该字节的格式如 下：

位号	内容
7	符号位
6	零位
5	0
4	辅助进位位

助记符	指令	二进制代码	位置

位号	内容
3	0
2	奇偶位
1	1
0	进位位

例如，如果进位位置为 1 而其余所有状态位都复位为 0，则状态位寄存器 的内容为如下字节：00 000 011。执行 PUSH 指令后，堆栈指针自动 减 2。

状态位：不受影响。

示例：假设执行 PUSH BC 。指令字节的格式如下：11 000 101。寄存器对 BC 的内容将按如下方式存入存储器：B 存入堆栈 指针地址减 1 的单元，C 存入堆栈指针地址减 2 的单元。随后堆栈 指针减 2。

助记符	指令	二进制代码	位置
POP	POP DATA OFF STACK	11 **(rp)**0 001	(Byte 1)

操作：指定寄存器对 (rp) 的内容从由堆栈指针指示的地址处的 两个存储字节中取出。位于堆栈指针地址处的存储字节的内容被装入该寄 存器对的第二个寄存器，位于堆栈指针地址加 1 处的字节内容被装入第一 个寄存器。

如果指定的是由状态位寄存器和累加器组成的寄存器对（PSW），则使用位 于堆栈指针地址加 1 处的字节内容，按照 PUSH 指令说明中给出的格 式来置位或复位各个状态位。

助记符	指令	二进制代码	位置
执行 POP 指令后，堆栈指针自动加 2。			
受影响的状态位：无，除非指定寄存器对 PSW。			
示例： PUSH 指令下的示例的逆操作可以说明 POP 指令的操 作。			

助记符	指令	二进制代码	位置
DAD	DOUBLE ADD	00 **(rp)**1 001	(Byte 1)
操作：由指定寄存器对 (rp) 中两个字节构成的 16 位数与由 `` H`` 和 L 寄存器中两个字节构成的 16 位数相加，结果存入 H 和 L 寄存器对。			
受影响的状态位：进位位。			
示例：假设寄存器对 BC 中两个字节构成的 16 位数为 00 101 1 11 01 111 111。假设 H 和 L 寄存器对中两个字节构成的 16 位 数为 01 100 000 00 100 101。指令 DAD BC``(00 **00**\ 1 001）将 把这两个数相加，并把结果存入 ``H 和 L 寄存器对。相加的结果 为：10 001 111 10 100 100。由于没有发生进位，进位位被复位为 0。			

助记符	指令	二进制代码	位置
INX	INCREMENT REGISTER PAIR	00 **(rp)**0 011	(Byte 1)
操作：由指定寄存器对 (rp) 中两个字节构成的 16 位数加 1。			
状态位：不受影响。			

助记符	指令	二进制代码	位置

示例：假设存在指令 00 100 011。根据寄存器对位模式表，由
H 和 L 寄存器对中两个字节构成的 16 位数将加 1。如果初始

16 位数为 10 001 111 10 100 100，则新的 16 位数为 10 001 111 10 1 00 101。

助记符	指令	二进制代码	位置
DCX	DECREMENT REGISTER PAIR	00 **(rp)**1 011	(Byte 1)

操作：由指定寄存器对中两个字节构成的 16 位数减 1。

状态位：不受影响。

示例：假设存在指令 00 101 011。根据寄存器对位模式表，由
H 和 L 寄存器对中两个字节构成的 16 位数将减 1。如果初始

16 位数为 10 001 111 10 100 101，则新的 16 位数为 10 001 111 10 1 00 100。

助记符	指令	二进制代码	位置
XTHL	EXCHANGE STACK	11 100 011	(Byte 1)

操作：L 寄存器中的字节与由堆栈指针寻址的存储字节交换。``H `` 寄存器中的字节与堆栈指针所寻址地址加 1 处的存储字节交换。

状态位：不受影响。

助记符	指令	二进制代码	位置
示例：`XCHG` 指令下的示例与执行 `XTHL` 指令时发生的操作类 似。			

助记符	指令	二进制代码	位置
`SPHL`	LOAD `SP` FROM `H` AND `L`	`11 111 001`	(Byte 1)
操作：`H` 和 `L` 寄存器的 16 位内容替换堆栈指针的内容，而 不影响 `H` 和 `L` 寄存器的内容。			
示例：假设 `H` 寄存器字节为 `10 001 111`，`L` 寄存器字节为 `10 000 011`。假设堆栈指针地址为 `00 001 100 01 111 111`。执行 `SPHL `` 指令将把堆栈指针装入：10 001 111 10 000 011。``H` 和 `L` 寄 存器的内容保持不变。			

D. 累加器循环移位指令

这是一组特殊的四条指令，仅适用于 ALTAIR 8800 的累加器。它们只需要一个 指令字节，并且不涉及存储器或寄存器的变化形式。

助记符	指令	二进制代码	位置
`RLC`	ROTATE ACCUMULATOR LEFT	`00 000 111`	(Byte 1)
操作：累加器字节向左循环移动一个位。第 `7` 位现在移动到第 `0` 位， 进位位被置为循环移位前第 `7` 位的值。			
受影响的状态位：进位位。			

助记符	指令	二进制代码	位置

示例：假设累加器字节为 `10 001 000`，并执行 `RLC` 指令。进位 位被置为等于累加器字节第 `7` 位的值（`1`），累加器的内容向左循环移动 一个位。第 `7` 位现在占据第 `0` 位：`00 010 001`。

助记符	指令	二进制代码	位置
`RRC`	ROTATE ACCUMULATOR RIGHT	`00 001 111`	(Byte 1)

操作：累加器字节向右循环移动一个位。第 `0` 位现在移动到第 `7` 位， 进位位被置为循环移位前第 `0` 位的值。

受影响的状态位：进位位。

示例：假设累加器字节为 `10 001 000`，并存在 RRC 指令。进位位被 置为等于累加器字节第 `0` 位的值（`0`），累加器的内容向右循环移动一个 位。第 `0` 位现在占据第 `7` 位：`01 000 100`。

助记符	指令	二进制代码	位置
`RAL`	ROTATE ACCUMULATOR LEFT THROUGH CARRY	`00 010 111`	(Byte 1)

操作：累加器字节通过进位位向左循环移动一个位。第 `7` 位随后占据 进位位，进位位占据第 `0` 位。

受影响的状态位：进位位。

示例：假设累加器字节为 `10 001 000`，进位位为 `1`，并执行 **`RAL`** 指令。累加器的内容通过进位位向左循环移动一个位。第 `7` 位现在占据

进位位（`1`），进位位现在占据第 `0` 位：`00 010 001`。

助记符	指令	二进制代码	位置
RAR	ROTATE ACCUMULATOR RIGHT THROUGH CARRY	00 011 111	(Byte 1)
操作：累加器字节通过进位位向右循环移动一个位。第 0 位现在占据 进位位，进位位占据第 7 位。			
受影响的状态位：进位位。			
示例：假设累加器字节为 10 001 000，进位位为 1，并执行 **RAR** 指令。累加器的内容通过进位位向右循环移动一个位。第 0 位现在占据 进位位，进位位现在占据第 7 位：11 000 100。			

E. 数据传送指令

在 ALTAIR 8800 中，数据可以方便地在寄存器之间传送，也可以在存储器与寄 存器之间传送。这些操作中有些是直接的数据传送，不涉及其他操作。例如， MOV 指令把一个字节的数据从一个寄存器（源寄存器）传送到另一个 寄存器（目的寄存器）。另一些数据传送则伴随着算术运算或逻辑运算。例如， ADD 指令把指定寄存器的内容加到累加器的内容上。

还有一类数据传送指令只涉及累加器以及 H 和 L 寄存器对。 例如，STA 指令用累加器的内容替换存放在指定存储地址处的数据字节。

本节讲述十五种独立的数据传送指令，但值得注意的是，许多其他指令同样涉及 数据传送（例如 PUSH、POP、DAD、XCHG、XTHL、 SPHL 等）。不过，为了本操作手册的高效组织，这些指令放在别处讲述 更为合适。

本节讲述的数据传送指令分为三个小节。第一小节是数据传送（MOV、 STAX 和 LDAX）。第二小节是寄存器/存储器到累加器的传送（ADD、 ADC、SUB、SBB、ANA、XRA、ORA 和 CMP）。第三小节是直接寻址传送（STA、LDA、SHLD 和 LHLD）。

1. 数据传送指令

数据传送指令有三种，每种都是无条件指令。每种指令至少有两个变体，变体由 程序员指定的寄存器或存储地址决定。

助记符	指令	二进制代码	位置
`MOV`	MOVE DATA	`01` **(DDD) (SSS)**	(Byte 1)

操作：`(SSS)``(源寄存器)的内容被传至 ``(DDD)``(目的寄存器)。 ``(SSS)` 的内容保持不变。源寄存器和目的寄存器的位模式如下：

寄存器	位模式
B	`000`
C	`001`
D	`010`
E	`011`
H	`100`
L	`101`
存储器引用 M	`110`
A	`111`

源寄存器和目的寄存器不能同时等于 `110`。

状态位：不受影响。

示例：假设需要把寄存器 `E` 的内容传送到累加器。通过查阅上面 给出的寄存器位模式表，可以构造出合适的 `MOV` 指令：`01 111 011`。

助记符	指令	二进制代码	位置
`STAX`	STORE ACCUMULATOR	`00 0`**`(X)`**`0 010`	(Byte 1)
操作：累加器的内容被存入由寄存器 `B` 和 `C` 或寄存器 `D` 和 `E` 指定的存储地址。第 4 位 `(X)` 为 `0` 时指定寄存器 `B` 和 `C`，为 `1` 时指定寄存器 `D` 和 `E`。			
状态位：不受影响。			
示例：假设需要把累加器的内容存入由寄存器 `D` 和 `E` 指定的存储 地址。合适的 `STAX` 指令是：`00 010 010`。			

助记符	指令	二进制代码	位置
`LDAX`	LOAD ACCUMULATOR	`00 0`**`(X)`**`1 010`	(Byte 1)
操作：由寄存器 `B` 和 `C` 或由寄存器 `D` 和 `E` 指定的 存储地址中的内容取代累加器的内容。第 4 位 `(X)` 为 `0` 时指定 寄存器 `B` 和 `C`，为 `1` 时指定寄存器 `D` 和 `E`。			
状态位：不受影响。			
示例：假设需要把由寄存器 `B` 和 `C` 指定的存储地址中的内容装入 累加器。合适的 `LDAX` 指令是：`00 001 010`。			

2. REGISTER/MEMORY TO ACCUMULATOR TRANSFERS

共有八条寄存器/存储器到累加器的传送指令，且每一条都是无条件的。这八条指令中的每一条都有八种变体，由程序员所指定的寄存器决定。下列是每个寄存器对应的位模式：

寄存器	位模式
B	000
C	001
D	010
E	011
H	100
L	101
存储器地址 M	110
A	111

其中四条指令涉及算术（加或减）运算，其余四条涉及逻辑运算。

助记符	指令	二进制代码	位置
ADD	**ADD** 寄存器/累加器到存储器	10 000	（**Byte 1**） **(reg)**

操作：指定寄存器 (reg) 的内容与累加器的内容相加。 受影响的标志位：进位位、符号位、零位、奇偶位与辅助进位位。 示例：假设需要将寄存器 B 的内容与累加器相加。参考上面给出的寄存 器位模式表，相应的指令为：10 000 000。若寄存器 B 与累加器中的数 据字节分别为 11 010 100 与 01 100 010，则将执行如下加法：

```
11 010 100   寄存器 B 字节

01 100 010   累加器 字节
```

助记符	指令	二进制代码	位置

```
                ———————

            100 110 110   新 累加器 字节
```

由于新的累加器字节有九位，进位位将被置为 1，以表示发生了进位。

助记符	指令	二进制代码	位置

`ADC` | **ADD** 寄存器/存储器与进位到累加器 | `10 001` | (**Byte 1**)
| **(reg)** |

操作：指定寄存器 `(reg)` 与进位位的内容都被加到累加器上。 受影响的标志位：进位位、符号位、零位、奇偶位与辅助进位位。 示例：假设需要将寄存器 `C` 与进位位的内容加到累加器上。参考上面给 出的寄存器位模式表，相应的指令为：`10 001 001`。若寄存器 `C` 与累加 器中的数据字节分别为 `00 100 011` 与 `01 011 100`，且进位位为 1，则将执行如 下加法：

```
            00 100 011   寄存器 C 字节

            01 011 100   累加器 字节

                   1   进位位

                ———————

            100 000 000   新 累加器 字节
```

若新的累加器字节有九位，多余的位会将进位位置为 1。

助记符	指令	二进制代码	位置

`SUB` | 从累加器减去寄存器/存储器 | `10 010` | (**Byte 1**) | **(reg)** |

操作：指定寄存器的内容从累加器的内容中减去。ALTAIR 8800 通过一种 称为二进制补码运算的简单加法过程来实现减法。若结果只有八位，则不存在进 位位。这意味着发生了借位，进位位被置为 `1`。注意，此操作与 `ADD` 指令中 的操作相反。 受影响的标志位：进位位、符号位、零位、奇偶位与辅助进位位。 示例：假设需要将累加器的内容清零。实现这一要求的一种有效方法是执行 一条 `SUB A` 指令（`10 010 111`），其中 `A` 指定了 SUB 指令中累加 器的变体。执行该指令将使累加器的内容减去其自身。

MNEMONIC	INSTRUCTION	BINARY CODE	POSITION
`SBB`	从累加器带借位减去 寄存器/存储器	`10 011` **(reg)**	(Byte 1)

操作：进位位的内容与指定寄存器的内容相加，然后利用 二进制补码运算从累加器中减去该结果。

受影响的标志位：进位、符号、零、奇偶和辅助进位。

示例：假定针对 `B` 变量的 `SBB` 指令已实现（`10 011 000`）。寄存器 `B` 的内容将与进位位相加，然后该结果从 累加器中减去。标志位将相应地被置位或复位。

MNEMONIC	INSTRUCTION	BINARY CODE	POSITION
`ANA`	逻辑与寄存器/存储器 与累加器	`10 100` **(reg)**	(Bytte 1)

操作：指定寄存器的内容与累加器的内容进行逻辑 `AND` 运算。进位位复位为 `0`。

MNEMONIC	INSTRUCTION	BINARY CODE	POSITION
受影响的标志位：进位、零、符号和奇偶。			
示例：假定寄存器 `L` 的内容为 `10 001 100`，且累加器 的内容为 `10 000 101`。`ANA` 指令的 `L` 变量（`10 100 101`）随后将使两个寄存器的内容按位相互进行 `AND` 运算。由于只有当两个位均为 `1` 时，两个位的逻辑 `AND` 运算结果才为 `1`，故发生以下过程： `10 001 100` 寄存器 L `10 000 101` 累加器 ——— `10 000 100` 寄存器 L `AND` 累加器			

MNEMONIC	INSTRUCTION	BINARY CODE	POSITION
`XRA`	逻辑异或寄存器/存储器 与累加器	`10 101` **(reg)**	(Byte 1)
操作：指定寄存器的内容与累加器的内容进行逻辑 `EXCLUSIVE-OR` 运算。进位位复位为 `0`。			
受影响的标志位：进位、零、符号和奇偶。			
示例：由于两个位的 `EXCLUSIVE-OR` 运算结果仅当两位取值 不同才为 `1`，因此 `XRA` 指令可用于将累加器清零。该功能 通过 `XRA` 指令与 `A` 变量实现。所得语句为 `10 101` **111**（参见上文给出的寄存器位模式表）。 `XRA` 指令还可用于监视已被指定为条件字节的某个字节中 各个位的状态。例如，假定某字节被指定用来记录八个独立的 真/假条件，其中 `1` 表示真，`0` 表示假。为了检查是否有任一 条件发			

MNEMONIC	INSTRUCTION	BINARY CODE	POSITION
生了变化，可将原始数据字节移入累加器，并与更新后的 数据字节进行 `EXCLUSIVE-OR` 运算。未改变的条件将产生 `0` 位，而改变的条件将产生 `1` 位。			

MNEMONIC	INSTRUCTION	BINARY CODE	POSITION
`ORA`	逻辑或寄存器/存储器 与累加器	`10 110` **(reg)**	(Byte 1)
操作：指定寄存器的内容与累加器的内容进行逻辑 `OR` 运算。进位位复位为零。			
受影响的标志位：进位、零、符号和奇偶。			
示例：由于只有当每个位的值均为 `0` 时，两个位的 `OR` 运算结果才为 `0`，因此 `ORA` 指令可用于将一组 位设置为一系列 `1`。			

MNEMONIC	INSTRUCTION	BINARY CODE	POSITION
`CMP`	比较寄存器/存储器与 累加器	`10 111` **(reg)**	(Byte 1)
操作：指定寄存器的内容与累加器的内容进行比较，即 用后者减去前者。寄存器与累加器的内容不受此操作影响， 标志位将相应地被置位或复位。			
受影响的标志位：进位、零、符号和奇偶（注：若一个 字节为正、另一个为负，则进位位的含义相反）。			
示例： `CMP` 指令在确定任一特定寄存器的内容何时 等于累加器的内容时很有用。若两个字节相等，减法 将得到 `0` 结果，零状态位将被置为 `1`。若寄存器内容 大于累加器内容，由于发生了减法，进位位将被置为 `1`。 若寄存器内容小于累加器内容，进位位将被复位为 `0`。			

3. 直接寻址指令

本节所述的四条指令用于把累加器以及 H 和 L 寄存器的内容存入存储器，或者把 来自存储器的数据装入累加器以及 H 和 L 寄存器。这四条指令都占用三个字节。 第一个字节是具体的指令，第二和第三个字节提供存储地址。

助记符	指令	二进制代码	位置
STA	STORE ACCUMULATOR DIRECT	00 110 010	(字节 1)
		(低地址)	(字节 2)
		(高地址)	(字节 3)

操作：累加器的内容被存入由第 2 和第 3 字节指定的存储地址。

状态位：不受影响。

示例：设累加器字节为 00 010 110，并有一条 STA 指令， 其存储地址如下：

```
01 000 000   (字节 2)

01 000 001   (字节 3)
```

然后，累加器字节将被存入该存储地址。

助记符	指令	二进制代码	位置
LDA	LOAD ACCUMULATOR DIRECT	00 111 010	(字节 1)

助记符	指令	二进制代码	位置
		(低地址)	(字节 2)
		(高地址)	(字节 3)
操作：累加器从该指令第 2 和第 3 字节给出的存储地址处装入字节内容。			
状态位：不受影响。			
STA 指令所述示例的逆过程可以说明 LDA 指令的操作。			

助记符	指令	二进制代码	位置
SHLD	STORE H AND L DIRECT	00 100 010	(字节 1)
		(低地址)	(字节 2)
		(高地址)	(字节 3)
操作：L 寄存器的内容被存入由第 2 和第 3 字节指定的存储地址。 H 寄存器的内容被存入下一个更高的存储地址。			
状态位：不受影响。			
示例：设 L 寄存器字节为 00 101 100，H 寄存器字节为 00 101 111，并有一条 SHLD 指令，其地址如下：			

助记符	指令	二进制代码	位置
		01 000 101 (字节 2)	
		01 110 101 (字节 3)	
然后，L 寄存器字节将被存入该存储地址，H 寄存器字节将被存入 下一个最高的地址。			

助记符	指令	二进制代码	位置
LHLD	LOAD H AND L DIRECT	00 101 010	(字节 1)
		(低地址)	(字节 2)
		(高地址)	(字节 3)
操作：L 寄存器从第 2 和第 3 字节给出的存储地址处装入字节内容。 H 寄存器从下一个更高的存储地址处装入字节内容。			
状态位：不受影响。			
SHLD 指令所述示例的逆过程可以说明 LHLD 指令的操作。			

F. 立即数指令

ALTAIR 8800 有十条立即数指令。这些指令使计算机处理一个或两个字 节的数据，这些数据是指令的组成部分。立即数指令可用于把两个字节的 数据装入指定的寄存器对、把一个字节的数据传送到指定的寄存器或存储 单元，以及用累加器的内容与一个字节的立即数执行算术和逻辑运算。

立即数数据的一个典型字节是数学常数，例如圆周率（π）。立即数也 可以是程序员指定的数或量，例如实际的或预计的库存数量。例如，使用 一条或多条立即数指令的程序，可以让计算机把某种产品的实际库存与期 望库存进行比较。当库存数量达到程序中指定的水平时，计算机可以通知程序员或操作员需要补货。

助记符	指令	二进制代码	位置
`LXI`	立即数装入寄存器对	`00` **(rp)**`0 001`	(字节 1)
		(Data)	(字节 2)
		(Data)	(字节 3)

操作：把两个字节的立即数装入由指令第 1 字节中的 `(rp)` 所指定 的寄存器对。第一个数据字节（最低有效 8 位）装入所指定寄存器对的 第二个寄存器，第二个数据字节（最高有效 8 位）装入所指定寄存器对 的第一个寄存器。如果指定的寄存器对是堆栈指针，则这个过程相反。 寄存器对的位模式如下：

状态位：不受影响。

示例：下面这条 `LXI` 指令被装入 RAM：

第 1 字节的位 4 和位 5 指定字节 2 和字节 3 中的数据要装入寄存 器 `D` 和 `E`。字节 2 装入 `D`，字节 3 装入 `E`。

助记符	指令	二进制代码	位置
`MVI`	传送立即数	`00` **(reg)** `110`	(字节 1)
		(Data)	(字节 2)

助记符	指令	二进制代码	位置

操作：把一个字节的立即数传送到指定的寄存器或存储器字节。适 用的寄存器位模式如下：

寄存器	位模式
B	000
C	001
D	010
E	011
H	100
L	101
存储器引用 M	110
A	111

状态位：不受影响。

示例：下面这条 `MVI` 指令被装入 RAM：

第 2 字节中的立即数（八进制 377）被传送到寄存器 `E`。

助记符	指令	二进制代码	位置
`ADI`	立即数与累加器相加	11 000 110	

助记符	指令	二进制代码	位置
			(字节 1)
		(Data)	(字节 2)

操作：把第 2 字节中的立即数加到累加器的内容上。

受影响的状态位：进位位、符号位、零位、奇偶位和辅助进位位。

示例：假设累加器字节为 11 110 000，且存在 `ADI` 指令。 `ADI` 指令中的立即数为 10 000 000。执行 `ADI` 指令后，累加 器中留下 01 110 000，进位位置 1。其余状态位都复位为 0。

助记符	指令	二进制代码	位置
`ACI`	立即数及进位与累加器相加	11 001 110	(字节 1)
		(Data)	(字节 2)

操作：把第 2 字节的数据和进位位的内容加到累加器的内容上。

受影响的状态位：进位位、符号位、零位、奇偶位和辅助进位位。

示例：假设累加器字节为 11 110 000，进位位置 1，且存在 `ACI` 指令。`ACI` 指令中的立即数为 00 101 100。执行 `ACI` 指令后，累加器中留下和数 00 011 101，进位位和奇偶位都置 1。其余状态位都复位为 0。

助记符	指令	二进制代码	位置
SUI	从累加器减去立即数	11 010 110	(字节 1)
		(Data)	(字节 2)

操作：用补码算术把第 2 字节的数据从累加器的内容中减去。由于 ALTAIR 8800 用加法来实现减法，因此如果没有发生进位，就意味着 发生了借位，进位位置 1。如果没有发生借位，则确实发生了进位，进 位位复位为 0。注意，这个操作与 ADI 或 ACI 指令中发生的 情况相反。

受影响的状态位：进位位、符号位、零位、奇偶位和辅助进位位。

示例：假设需要从累加器中减去 00 000 100。所得指令如下：

如果累加器字节为 00 001 010，执行 SUI 指令后累加器中留下 00 000 110。由于这是减法操作且没有进位，说明发生了借位，进位位 置 1。奇偶位也置 1，其余状态位都复位为 0。

助记符	指令	二进制代码	位置
SBI	从累加器减去立即数及进位	11 011 110	(字节 1)
		(Data)	(字节 2)

操作：把第 2 字节的数据与进位位的内容相加，再用补码算术从累 加器中减去该结果。

受影响的状态位：进位位、符号位、零位、奇偶位和辅助进位位。

助记符	指令	二进制代码	位置

示例：假设需要执行 `SBI` 指令。数据字节的内容将与进位位相 加，再从累加器中减去该结果。由于这是减法操作，如果没有发生进位 （意味着发生了借位），进位位置 `1`；如果发生了进位（意味着没有发 生借位），进位位复位为 `0`。

助记符	指令	二进制代码	位置
`ANI`	立即数与累加器逻辑与	`11 100 110`	(字节 1)
		(Data)	(字节 2)

操作：把数据字节的内容与累加器的内容逐位进行逻辑 `AND`。 进位位复位为 `0`。

受影响的状态位：进位位、符号位、零位和奇偶位。

示例：假设数据字节的内容为 `00 111] 011`，累加器的内容为 `11 101 110`。那么 `ANI` 指令将使两个字节的内容逐位进行 `AND`。因为两位的逻辑 `AND` 只有在两位都为 `1` 时才为 `1`，所 以发生如下过程：

助记符	指令	二进制代码	位置
`XRI`	立即数与累加器逻辑异或	`11 101 110`	(字节 1)
		(Data)	(字节 2)

操作：把指令第 `2` 字节的数据与累加器字节进行 `EXCLUSIVE-OR` 运算。进位位复位为 `0`。

助记符	指令	二进制代码	位置
受影响的状态位：进位位、符号位、零位和奇偶位。			
示例：一位与 0 进行 EXCLUSIVE-OR 时不变，与 1 进行 EXCLUSIVE-OR 时取反。因此 EXCLUSIVE-OR 功能可以用来取 反累加器中任意一位或所有位。例如，要取反累加器中除位置 7 的位以 外的所有位，需要如下数据字节：01 111 111。如果累加器字节为 10 110 001，执行 XRI 指令时会发生如下操作：			

助记符	指令	二进制代码	位置
ORI	立即数与累加器逻辑或	11 110 110	(字节 1)
		(Data)	(字节 2)
操作：把指令第 2 字节的数据与累加器字节进行逻辑 OR 运 算。进位位复位为 0。			
受影响的状态位：进位位、符号位、零位和奇偶位。			
示例：ORI 指令可用来给累加器加 1。假设累加器字节为 10 000 100，且存在 ORI 指令。由于两位的 OR 运算只有在两位都 为 0 时才产生 0，所以如果位置 0 的位为 0，数据字节 00 000 001 就 会给累加器加 1。否则累加器字节不变。			

助记符	指令	二进制代码	位置
CPI	立即数与累加器比较	11 111 110	(字节 1)
		(Data)	(字节 2)

助记符	指令	二进制代码	位置

操作：把指令第 2 字节的数据与累加器的内容进行比较，方法是后 者减去前者。累加器和数据字节的内容不受此操作影响，状态位按相应 情况置位或复位。

受影响的状态位：进位位、零位、符号位、奇偶位和辅助进位位。

示例：`CPI` 指令在判断累加器内容何时等于数据字节内容时很 有用。如果两个字节相等，减法过程将得到 0 结果，零状态位置 1。如 果数据字节内容大于累加器内容，由于发生了减法，进位位置 1。如果 数据字节内容小于累加器内容，进位位复位为 0。

G. 转移指令

ALTAIR 8800 具有强大的转移能力。转移使计算机能够从程序中的一步跳到另一 步。转移还允许计算机从存储器中调用一组指定的指令并将其插入程序。返回功能 则使计算机在执行完指定的指令集后恢复正常运行。

转移是计算机最重要的能力之一。例如，从程序中的一点跳到另一点可以节省时 间，而从存储器中调用一组特殊的指令意味着频繁使用的指令序列只需存储在存储 器中的一个位置即可。这能显著提高计算机的处理速度和效率。转移还提高了计算 机的经济性，因为完成复杂程序所需的存储器更少。此外，从存储器中调用频繁使 用的指令集的能力还可以节省大量的编程时间。

术语“子程序”用于描述存储在存储器中的一组特殊指令。典型的子程序可能包括计 算三角函数和平方根或进行复杂逻辑比较的指令集。这些子程序每一个都可能相当 长。如果一个程序需要十几次或更多三角函数运算以及若干次平方根提取，显然使 用子程序可以节省大量的编程时间和存储空间。

转移指令可以是有条件的或无条件的。条件转移意味着只有在满足指定条件时才执 行特定的转移操作。例如，典型的条件转移指令是 `CZ`（CALL IF ZERO）。如果在处理 `CZ` 指令时零位确实为零，程序计数器将自动移 到程序中跟随 `CZ` 指令的两个地址字节所指定的存储地址。

无条件转移则无需满足某些指定条件即可发生。

转移指令每条指令占用一个或三个字节。第一个字节是实际的指令，第二个和第三 个字节则分别是低和高存储地址。地址字节告诉程序计数器移到哪里。只需一个字 节的指令不需要存储地址，因为该字节中的某些位会将程序计数器指向某些寄存器 或堆栈指针，二者之一含有必要的寻址信息。

1. 跳转指令

跳转指令可以有条件地或无条件地改变程序的正常执行顺序。例如，一个程序可能 包含这样一组指令：仅当前一个运算的结果大于零时才执行。然而，如果结果为 零，这组指令就变得多余且无必要。因此，程序中包含一条 `JUMP` 语句，指示计 算机前进到该指令集之后的任意指定地址。由于只有当前一个运算的结果为零时才 执行跳转，因此这是一种条件转移操作。这条特定指令的机器语言助记符是 `JZ`（JUMP IF ZERO）。

十条 `JUMP` 指令中除一条之外都需要三个字节。第一个字节是特定的机器语言指 令，第二个和第三个字节则分别是程序中待选中部分的低和高存储地址—如果执行 跳转，程序计数器将选中该部分。`PCHL` 指令只需要最初的机器语言指令字节， 因为程序要跳转到的存储位置是计算机已知的。在这种情况下，这些存储位置恰好 就是 `H` 和 `L` 寄存器，它们的内容会被送入程序计数器。

除 `PCHL` 和 `JMP` 指令外，所有 `JUMP` 指令都是有条件的。如果指定条件为 真，程序计数器会自动前进到指令指定的地址。如果指定条件不为真，程序将继续 顺序执行，不会发生跳转。

助记符	指令	二进制代码	位置
`PCHL`	LOAD PROGRAM COUNTER	`11 101 001`	第 1 字节

操作：程序计数器跳到由 ``H``和 ``L``寄存器内容指定的存储地址。程序计 数器的最高有效 8 位装入 ``H``寄存器的内容，最低有效 8 位装入 ``L``寄存器 的内容。

状态位：不受影响。

示例：假设 ``H``和 ``L``寄存器的内容如下：

```
H:                                      10 111 000

L:                                      11 010 110
```

指令 ``PCHL``会自动将该存储地址传送到程序计数器，如下所示：

助记符	指令	二进制代码	位置
		最高有效位	最低有效位
	程序计数器：	10 111 000	11 010 110
	现在程序将跳转到程序计数器中指定的新地址，然后继续执行。		

助记符	指令	二进制代码	位置
JMP	JUMP	11 000 011	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节
	操作：程序计数器无条件跳到字节 2 和字节 3 指定的存储地址，程序从新 位置继续执行。		
	状态位：不受影响。		

助记符	指令	二进制代码	位置
示例：假设 ``JMP``指令和地址位模式如下： 11 000 011 第 1 字节 10 111 000 第 2 字节 11 010 110 第 3 字节 程序计数器将跳到由字节 2 和字节 3 指定的存储地址，程序执行将从新地址继 续。			

助记符	指令	二进制代码	位置
`JC`	JUMP IF CARRY	11 011 010	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节
操作：这是一条条件指令。如果进位位的状态为 1，表示发生了进位，程序 计数器跳到字节 2 和字节 3 指定的地址，程序执行然后从新地址继续。如果进 位位为 0，表示没有发生进位，程序继续顺序执行。			
状态位：不受影响。			
示例：假设进位位为 1 且存在一条 ``JC``指令。程序计数器将跳到字节 2 和字节 3 指定的地址，程序将从新地址继续执行。			

助记符	指令	二进制代码	位置
`JNC`	JUMP IF NO CARRY	11 010 010	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节
操作：这是一条条件指令。如果进位位的状态为 0，表示没有发生进位，程 序计数器跳到字节 2 和字节 3 指定的地址，程序执行然后从新地址继续。如果 进位位为 1，表示发生了进位，程序继续顺序执行。			
状态位：不受影响。			
示例：``JC``指令下所述示例的反例可以说明 ``JNC``指令的操作。			

助记符	指令	二进制代码	位置
`JZ`	JUMP IF ZERO	11 001 010	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节
操作：这是一条条件指令。如果零位的状态为 1，表示存在零，程序计数器 跳到字节 2 和字节 3 指定的地址，程序执行然后从新地址继续。如果零位为 0，表示不存在零，程序继续顺序执行。			

助记符	指令	二进制代码	位置
状态位：不受影响。			
示例：假设零位为 1（存在零）且存在一条 ``JZ``指令。程序计数器将跳到 字节 2 和字节 3 指定的地址，程序将从新地址继续执行。			

助记符	指令	二进制代码	位置
JNZ	JUMP IF NOT ZERO	11 000 010	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节
操作：这是一条条件指令。如果零位的状态为 0（不存在零）且存在一条 ``JNZ``指令，程序计数器跳到字节 2 和字节 3 指定的地址，程序执行然后从新 地址继续。如果零位为 1，表示存在零，程序继续顺序执行。			
状态位：不受影响。			
示例：``JZ``指令下所述示例的反例可以说明 ``JNZ``指令的操作。			

助记符	指令	二进制代码	位置
JM	JUMP IF MINUS	11 111 010	第 1 字节
		低存储地址	

助记符	指令	二进制代码	位置
			第 2 字节
		高存储地址	第 3 字节
操作：这是一条条件指令。如果符号位的状态为 1（结果为负），程序计数 器跳到字节 2 和字节 3 指定的地址，程序执行然后从新地址继续。如果符号位 为 0，结果为正，程序继续顺序执行。			
状态位：不受影响。			
示例：假设符号位为 1，表示结果为负，且存在 ``JM``指令。程序计数器将 跳到指令中字节 2 和字节 3 指定的地址，程序将从新地址继续执行。			

助记符	指令	二进制代码	位置
JP	JUMP IF PLUS	11 110 010	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节
操作：这是一条条件指令。如果符号位的状态为 0（结果为正），程序计数 器跳到字节 2 和字节 3 指定的地址，程序执行然后从新地址继续。如果符号位 为 1，结果为负，程序继续顺序执行。			
状态位：不受影响。			

助记符	指令	二进制代码	位置
示例：``JM``指令下所述示例的反例可以说明 ``JP``指令的操作。			

助记符	指令	二进制代码	位置
`JPE`	JUMP IF PARITY IS EVEN	`11 101 010`	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节
操作：这是一条条件指令。如果奇偶位的状态为 `1`（结果为偶校验），程序 计数器跳到字节 `2` 和字节 `3` 指定的地址，程序执行然后从新地址继续。如果奇 偶位为 `0`，奇偶性为奇，程序继续顺序执行。			
状态位：不受影响。			
示例：假设奇偶位为 `1`，表示结果为偶校验，且存在 ``JPE``指令。程序计数 器将跳到字节 `2` 和字节 `3` 指定的地址，程序将从新地址继续执行。			

助记符	指令	二进制代码	位置
`JPO`	JUMP IF PARITY IS ODD	`11 100 010`	第 1 字节
		低存储地址	第 2 字节

助记符	指令	二进制代码	位置
		高存储地址	第 3 字节
操作：这是一条条件指令。如果奇偶位的状态为 0（结果为奇校验），程序 计数器跳到字节 2 和字节 3 指定的地址，程序执行然后从新地址继续。如果奇 偶位为 1，奇偶性为偶，程序继续顺序执行。			
状态位：不受影响。			
示例：``JPE``指令下所述示例的反例可以说明 ``JPO``指令的操作。			

2. 调用指令

`CALL` 指令使程序执行存储在存储器中指定位置的子程序。`CALL` 指令可以是有 条件的或无条件的。许多子程序是以无条件方式调用的。例如，提取平方根的计算 序列相当长。在需要频繁提取平方根的程序中，编写一个单独的平方根提取子程序 可以节省大量的编程时间和存储空间。然后可以将该子程序存储在存储器中，由程 序在每次需要时调用。

条件 `CALL` 指令同样可用。它们提供了极大的灵活性，因为程序员可以指示计算 机在任意指定点对程序的状态做出逻辑判断。这样，满足指定条件时就可以调用子 程序。

子程序执行完毕后，程序计数器通过一条特殊的 `RETURN` 指令返回主程序中的下 一步。该指令在下一节中介绍。

所有 `CALL` 指令都需要三个字节。第一个字节是特定的机器语言指令，第二个和 第三个字节则分别是子程序第一条指令的低和高存储地址。

助记符	指令	二进制代码	位置
`CALL`	CALL	11 001 101	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节

助记符	指令	二进制代码	位置
		高存储地址	第 3 字节
操作：程序计数器无条件移到字节 2 和字节 3 指定的存储地址，然后执行 位于新位置的子程序。			
状态位：不受影响。			
示例：假设 ``CALL``指令和地址位模式如下： 11 001 101 第 1 字节 10 101 111 第 2 字节 11 111 010 第 3 字节 程序计数器将移到由字节 2 和字节 3 指定的存储地址，然后执行位于该处的子 程序。			

助记符	指令	二进制代码	位置
CC	CALL IF CARRY	11 011 100	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节

助记符	指令	二进制代码	位置
操作：这是一条条件指令。如果进位位的状态为 1，表示发生了进位，程序 计数器移到字节 2 和字节 3 指定的地址，然后执行位于该处的子程序。如果进 位位为 0，表示没有发生进位，程序继续顺序执行。			
状态位：不受影响。			
示例：假设进位位为 1 且存在 ``CC``指令。程序计数器将跳到字节 2 和字 节 3 指定的地址，然后执行位于该处的子程序。			

助记符	指令	二进制代码	位置
CNC	CALL IF NO CARRY	11 010 100	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节
操作：这是一条条件指令。如果进位位的状态为 0，表示没有发生进位，程 序计数器移到字节 2 和字节 3 指定的地址，然后执行位于该处的子程序。如果 进位位为 1，表示发生了进位，程序计数器继续顺序执行。			
状态位：不受影响。			
示例：``CC``指令下所述示例的反例可以说明 ``CNC``指令的操作。			

助记符	指令	二进制代码	位置
CZ	CALL IF ZERO	11 001 100	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节
操作：这是一条条件指令。如果零状态位的状态为 1，表示存在零，程序计 数器移到字节 2 和字节 3 指定的地址，然后执行位于该处的子程序。如果零状 态位为 0，表示不存在零，程序继续顺序执行。			
状态位：不受影响。			
示例：假设零状态位为 1 且存在 ``CZ``指令。程序计数器将移到字节 2 和 字节 3 指定的地址，然后执行位于该处的子程序。			

助记符	指令	二进制代码	位置
CNZ	CALL IF NOT ZERO	11 000 100	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节

助记符	指令	二进制代码	位置
操作：这是一条条件指令。如果零状态位的状态为 0，表示不存在零，程序 计数器移到字节 2 和字节 3 指定的地址，然后执行位于该处的子程序。如果零 状态位为 1，表示存在零，程序继续顺序执行。			
状态位：不受影响。			
示例：``CZ``指令下所述示例的反例可以说明 ``CNZ``指令的操作。			

助记符	指令	二进制代码	位置
`CM`	CALL IF MINUS	11 111 100	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节
操作：这是一条条件指令。如果符号位的状态为 1（结果为负），程序计数 器移到字节 2 和字节 3 指定的地址，然后执行位于该处的子程序。如果符号位 为 0，结果为正，程序继续顺序执行。			
状态位：不受影响。			
示例：假设符号位为 1 且存在 ``CM``指令。程序计数器将移到字节 2 和字 节 3 指定的地址，然后执行位于该处的子程序。			

助记符	指令	二进制代码	位置
CP	CALL IF PLUS	11 110 100	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节
操作：这是一条条件指令。如果符号位的状态为 0（结果为正），程序计数 器移到字节 2 和字节 3 指定的地址，然后执行位于该处的子程序。如果符号位 为 1，结果为负，程序继续顺序执行。			
状态位：不受影响。			
示例：``CM``指令下所述示例的反例可以说明 ``CP``指令的操作。			

助记符	指令	二进制代码	位置
CPE	CALL IF PARITY IS EVEN	11 101 100	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节
操作：这是一条条件指令。如果奇偶位的状态为 1（结果为偶校验），程序 计数器移到字节 2 和字节 3 指定的地址，然后执行位于该处的子程序。如果奇 偶位为 0，即奇校验，程序继续顺序执行。			

助记符	指令	二进制代码	位置
状态位：不受影响。			
示例：假设奇偶位的状态为 1 且存在 ``CPE``指令。程序计数器将移到字节 2 和字节 3 指定的地址，然后执行位于该处的子程序。			

助记符	指令	二进制代码	位置
CPO	CALL IF PARITY IS ODD	11 100 100	第 1 字节
		低存储地址	第 2 字节
		高存储地址	第 3 字节
操作：这是一条条件指令。如果奇偶位的状态为 0（结果为奇校验），程序 计数器移到字节 2 和字节 3 指定的地址，然后执行位于该处的子程序。如果奇 偶位为 1，即偶校验，程序继续顺序执行。			
状态位：不受影响。			
示例：``CPE``指令下所述示例的反例可以说明 ``CPO``指令的操作。			

3．返回指令

执行 CALL 子程序指令时，程序中下一条顺序指令的地址会自动压入堆栈。子程 序可以有一条或多条 RETURN 语句。大多数子程序的末尾都包含一条无条件 RETURN 指令。该指令将 CALL 指令压入堆栈的最后一个地址从堆栈弹出并送 入程序计数器。子程序执行完毕后，程序从最初的 CALL 子程序指令之后的地址 恢复顺序执行。

条件 `RETURN` 指令可以分散在子程序的各处。如果满足所需条件，程序就按上述 方式恢复顺序执行。

由于程序计数器接收到 `RETURN` 指令后返回的程序地址已经存储在堆栈中， `RETURN` 指令只需要一个字节。该字节的最后一位对于无条件 `RETURN` 为 `1`， 对于条件 `RETURN` 为 `0`。

助记符	指令	二进制代码	位置
`RET`	RETURN	`11 001 001`	第 1 字节

操作：子程序完成，程序计数器自动并无条件地返回到最初的 ``CALL``子程 序指令之后的下一个地址。

状态位：不受影响。

示例：假设 ALTAIR 8800 程序中的两条指令语句如下：

```
CALL        11 001 101          第 1 字节
                                第 2 字节
            （低存储地址）
                                第 3 字节
            （高存储地址）
CMA         00 101 111          第 1 字节
```

接收到 `CALL``指令时，程序计数器移到字节 2 和字节 3 指定的存储地址。同 时，下一条顺序指令（``CMA`）的地址被压入堆栈。

子程序中的最后一条指令必须是无条件``RETURN``（仅当希望返回时）。子程序执 行完毕并到达 `RET``指令时，程序计数器自动从堆栈接收主程序中下一条指令的 地址（``CMA`），并恢复顺序执行。

助记符	指令	二进制代码	位置
RC	RETURN IF CARRY	11 011 000	第 1 字节

操作：这是一条条件指令，可以插入到子程序末尾之前。如果进位位的状态 为 1，表示发生了进位，程序计数器自动返回到主程序中最初的 ``CALL``子程序 指令之后的下一个顺序地址。

状态位：不受影响。

示例：假设子程序中的三条指令如下：

RAL	00 010 111	第 1 字节
RC	11 011 000	第 1 字节
STAX	00 000 010	第 1 字节

到达 ``RC``指令时，如果进位位的状态为 1，表示发生了进位，程序计数器自动 返回到主程序中最初的 ``CALL``子程序指令之后的下一个顺序地址。如果进位位 的状态为 0，子程序通过执行 ``STAX``指令继续顺序执行。

助记符	指令	二进制代码	位置
RNC	RETURN IF NO CARRY	11 010 000	第 1 字节

操作：这是一条条件指令，可以插入到子程序末尾之前。如果进位位的状态 为 0，表示没有发生进位，程序计数器自动返回到主程序中最初的 ``CALL``子程 序指令之后的下一个顺序地址。如果进位位的状态为 1，表示发生了进位，子程 序继续顺序执行。

状态位：不受影响。

助记符	指令	二进制代码	位置

示例：``RC``指令下所述示例的反例可以说明 ``RNC``指令的操作。

助记符	指令	二进制代码	位置
`RZ`	RETURN IF ZERO	`11 001 000`	第 1 字节

操作：这是一条条件指令，可以插入到子程序末尾之前。如果零状态位的状 态为 `1`，表示存在 `0`，程序计数器自动返回到主程序中最初的 ``CALL``子程序指 令之后的下一个顺序地址。如果零状态位的状态为 `0`，表示不存在零，子程序继 续顺序执行。

状态位：不受影响。

示例：假设子程序中的三条指令如下：

ADD	`10 000 101`	第 1 字节
RZ	`11 001 000`	第 1 字节
LDAX	`00 011 010`	第 1 字节

到达 ``RZ``指令时，如果零状态位的状态为 `1`，表示结果为零，程序计数器自动 返回到主程序中最初的 ``CALL``指令之后的下一个顺序地址。如果零状态位的状 态为 `0`，子程序通过执行 ``LDAX``指令继续执行。

助记符	指令	二进制代码	位置
`RNZ`	RETURN IF NOT ZERO	`11 000 000`	第 1 字节

助记符	指令	二进制代码	位置

操作：这是一条条件指令，可以插入到子程序末尾之前。如果零状态位的状 态为 0，表示结果不为零，程序计数器自动返回到主程序中最初的 ``CALL``子程 序指令之后的下一个顺序地址。如果零状态位的状态为 1，表示结果为零，子程 序继续顺序执行。

状态位：不受影响。

示例：``RZ``指令下所述示例的反例可以说明 ``RNZ``指令的操作。

助记符	指令	二进制代码	位置
RM	RETURN IF MINUS	11 111 000	第 1 字节

操作：这是一条条件指令，可以插入到子程序末尾之前。如果符号位的状态 为 1（结果为负），程序计数器自动返回到主程序中最初的 ``CALL``子程序指令 之后的下一个顺序地址。如果符号位的状态为 0（结果为正），子程序继续顺序 执行。

状态位：不受影响。

示例：假设子程序中的三条指令如下：

```
SUB     10 010 001          第 1 字节

RM      11 111 000          第 1 字节

LDAX    00 011 010          第 1 字节
```

到达 ``RM``指令时，如果符号位的状态为 1，表示结果为负，程序计数器自动返 回到主程序中最初的 ``CALL``子程序指令之后的下一个顺序地址。如果符号位的 状态为 0，子程序通过执行 ``LDAX``指令继续顺序执行。

助记符	指令	二进制代码	位置
RP	RETURN IF PLUS	11 110 000	第 1 字节

操作：这是一条条件指令，可以插入到子程序末尾之前。如果符号位的状态 为 0（结果为正），程序计数器自动返回到程序中最初的 ``CALL``子程序指令之 后的下一个顺序地址。如果符号位的状态为 1（结果为负），子程序继续顺序执 行。

状态位：不受影响。

示例：``RM``指令下所述示例的反例可以说明 ``RP``指令的操作。

助记符	指令	二进制代码	位置
RPE	RETURN IF PARITY IS EVEN	11 101 000	第 1 字节

操作：这是一条条件指令，可以插入到子程序末尾之前。如果奇偶位的状态 为 1（结果为偶校验），程序计数器自动返回到主程序中最初的 ``CALL``子程序 指令之后的下一个顺序地址。如果奇偶位的状态为 0（结果为奇校验），子程序 继续顺序执行。

状态位：不受影响。

示例：假设子程序中的三条指令如下：

CMP	10 111 001	第 1 字节
RPE	11 101 000	第 1 字节
RLC	00 000 111	第 1 字节

助记符	指令	二进制代码	位置
到达 ``RPE``指令时，如果奇偶位的状态为 1，表示结果的奇偶性为偶，程序计数 器自动返回到主程序中最初的 ``CALL``子程序指令之后的下一个顺序地址。如果 奇偶性为奇，子程序通过执行 ``RLC``指令继续顺序执行。			

助记符	指令	二进制代码	位置
RPO	RETURN IF PARITY IS ODD	11 100 000	第 1 字节
操作：这是一条条件指令，可以插入到子程序末尾之前。如果奇偶位的状态 为 0（结果为奇校验），程序计数器自动返回到主程序中最初的 ``CALL``子程序 指令之后的下一个顺序地址。如果奇偶位的状态为 1（结果为奇校验），子程序 继续顺序执行。			
状态位：不受影响。			
示例：``RPE``指令下所述示例的反例可以说明 ``RPO``指令的操作。			

附录 A：ALTAIR 8800 指令集

定义：

DDD	目标寄存器
SSS	源寄存器
rp	寄存器对

寄存器编码

寄存器（**SSS** 或 **DDD**）	位模式
B	`000`
C	`001`
D	`010`
E	`011`
H	`100`
L	`101`
存储器	`110`
累加器	`111`

寄存器编码

寄存器对	位模式
B 和 C	00
D 和 E	01
H 和 L	10
SP	11

A. 命令指令

1. 输入/输出指令

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
IN	2	3	11 011 011	333	DB
OUT	2	3	11 010 011	323	D3

2. 中断指令

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
EI	1	1	11 111 011	373	FB
DI	1	1	11 110 011	363	F3

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
HLT	1	1	01 110 110	166	76
RST	1	3	11 (exp) 111	3(exp)7	
RST 0			11 000 111	3 0 7	C7
RST 1			11 001 111	3 1 7	CF
RST 2			11 010 111	3 2 7	D7
RST 3			11 011 111	3 3 7	DF
RST 4			11 100 111	3 4 7	E7
RST 5			11 101 111	3 5 7	EF
RST 6			11 110 111	3 6 7	F7
RST 7			11 111 111	3 7 7	FF

3. 进位位指令

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
CMC	1	1	00 111 111	077	3F

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
STC	1	1	00 110 111	067	37

4. 空操作指令

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
NOP	1	1	00 000 000	000	00

B. 单寄存器指令

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
INR	1	3	00 (DDD) 100	0(DDD)4	
INR B			00 000 100	0 0 4	04
INR C			00 001 100	0 1 4	0C
INR D			00 010 100	0 2 4	14
INR E			00 011 100	0 3 4	1C
INR H			00 100 100	0 4 4	24

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
INR L			00 101 100	0 5 4	2C
INR M			00 110 100	0 6 4	34
INR A			00 111 100	0 7 4	3C
DCR	1	3	00 (DDD) 101	0(DDD)5	
DCR B			00 000 101	0 0 5	05
DCR C			00 001 101	0 1 5	0D
DCR D			00 010 101	0 2 5	15
DCR E			00 011 101	0 3 5	1D
DCR H			00 100 101	0 4 5	25
DCR L			00 101 101	0 5 5	2D
DCR M			00 110 101	0 6 5	35
DCR A			00 111 101	0 7 5	3D
CMA	1	1	00 101 111	057	2F
DAA	1	1	00 100 111	047	27

C. 寄存器对指令

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
PUSH	1	3	11 (rp)0 101	3(rp)5	
PUSH B			11 000 101	3 0 5	C5
PUSH D			11 010 101	3 2 5	D5
PUSH H			11 100 101	3 4 5	E5
PUSH PSW			11 110 101	3 6 5	F5
POP	1	3	11 (rp)0 001	3(rp)1	
POP B			11 000 001	3 0 1	C1
POP D			11 010 001	3 2 1	D1
POP H			11 100 001	3 4 1	E1
POP PSW			11 110 001	3 6 1	F1
DAD	1	3	00 (rp)1 001	0(rp)1	
DAD B			00 001 001	3 1 1	09
DAD D			00 011 001	3 3 1	19

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
DAD H			00 101 001	3 5 1	29
DAD SP			00 111 001	3 7 1	39
INX	1	1	00 (rp)0 011	0(rp)3	
INX B			00 000 011	0 0 3	03
INX D			00 010 011	0 2 3	13
INX H			00 100 011	0 4 3	23
INX SP			00 110 011	0 6 3	33
DCX	1	1	00 (rp)1 011	0(rp)3	
DCX B			00 001 011	0 1 3	0B
DCX D			00 011 011	0 3 3	1B
DCX H			00 101 011	0 5 3	2B
DCX SP			00 111 011	0 7 3	3B
XCHG	1	1	11 101 011	353	EB
XTHL	1	5	11 100 011	343	E3

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
SPHL	1	1	11 111 001	371	F9

D. 累加器循环移位指令

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
RLC	1	1	00 000 111	007	07
RRC	1	1	00 001 111	017	0F
RAL	1	1	00 010 111	027	17
RAR	1	1	00 011 111	037	1F

E. 数据传送指令

1. 数据传送指令

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
MOV	1	1 or 2	01 (DDD) (SSS)	1(DDD)(SSS)	
MOV B,B			01 000 000	1 0 0	40

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
MOV B,C			01 000 001	1 0 1	41
MOV B,D			01 000 010	1 0 2	42
MOV B,E			01 000 011	1 0 3	43
MOV B,H			01 000 100	1 0 4	44
MOV B,L			01 000 101	1 0 5	45
MOV B,M			01 000 110	1 0 6	46
MOV B,A			01 000 111	1 0 7	47
MOV C,B			01 001 000	1 1 0	48
MOV 指令还有 46 种其他变体。（总计：64）					
MOV M,A			01 110 111	1 6 7	77
MOV A,B			01 111 000	1 7 0	78
MOV A,C			01 111 001	1 7 1	79
MOV A,D			01 111 010	1 7 2	7A
MOV A,E			01 111 011	1 7 3	7B

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
MOV A,H			01 111 100	1 7 4	7C
MOV A,L			01 111 101	1 7 5	7D
MOV A,M			01 111 110	1 7 6	7E
MOV A,A			01 111 111	1 7 7	7F
STAX	1	2	00 0(X)0 010	0(X)2	
STAX B			00 000 010	0 0 2	02
STAX D			00 010 010	0 2 2	12
LDAX	1	2	00 0(X)1 010	0(X)2	
LDAX B			00 001 010	0 1 2	0A
LDAX D			00 011 010	0 3 2	1A

备注

寄存器对 B 和 C–X 处为 0

寄存器对 D 和 E–X 处为 1

2. 寄存器/存储器到累加器传送

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
ADD	1	1	10 000 (SSS)	20(SSS)	
ADD B			10 000 000	2 0 0	80
ADD C			10 000 001	2 0 1	81
ADD D			10 000 010	2 0 2	82
ADD E			10 000 011	2 0 3	83
ADD H			10 000 100	2 0 4	84
ADD L			10 000 101	2 0 5	85
ADD M			10 000 110	2 0 6	86
ADD A			10 000 111	2 0 7	87
ADC	1	1	10 001 (SSS)	21(SSS)	
ADC B			10 001 000	2 1 0	88
ADC C			10 001 001	2 1 1	89
ADC D			10 001 010	2 1 2	8A

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
ADC E			10 001 011	2 1 3	8B
ADC H			10 001 100	2 1 4	8C
ADC L			10 001 101	2 1 5	8D
ADC M			10 001 110	2 1 6	8E
ADC A			10 001 111	2 1 7	8F
SUB	1	1	10 010 (SSS)	22(SSS)	
SUB B			10 010 000	2 2 0	90
SUB C			10 010 001	2 2 1	91
SUB D			10 010 010	2 2 2	92
SUB E			10 010 011	2 2 3	93
SUB H			10 010 100	2 2 4	94
SUB L			10 010 101	2 2 5	95
SUB M			10 010 110	2 2 6	96
SUB A			10 010 111	2 2 7	97

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
SBB	1	1	10 011 (SSS)	23(SSS)	
SBB B			10 011 000	2 3 0	98
SBB C			10 011 001	2 3 1	99
SBB D			10 011 010	2 3 2	9A
SBB E			10 011 011	2 3 3	9B
SBB H			10 011 100	2 3 4	9C
SBB L			10 011 101	2 3 5	9D
SBB M			10 011 110	2 3 6	9E
SBB A			10 011 111	2 3 7	9F
ANA	1	1	10 100 (SSS)	24(SSS)	
ANA B			10 100 000	2 4 0	A0
ANA C			10 100 001	2 4 1	A1
ANA D			10 100 010	2 4 2	A2
ANA E			10 100 011	2 4 3	A3

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
ANA H			10 100 100	2 4 4	A4
ANA L			10 100 101	2 4 5	A5
ANA M			10 100 110	2 4 6	A6
ANA A			10 100 111	2 4 7	A7
XRA	1	1	10 101 (SSS)	25(SSS)	
XRA B			10 101 000	2 5 0	A8
XRA C			10 101 001	2 5 1	A9
XRA D			10 101 010	2 5 2	AA
XRA E			10 101 011	2 5 3	AB
XRA H			10 101 100	2 5 4	AC
XRA L			10 101 101	2 5 5	AD
XRA M			10 101 110	2 5 6	AE
XRA A			10 101 111	2 5 7	AF
ORA	1	1	10 110 (SSS)	26(SSS)	

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
ORA B			10 110 000	2 6 0	B0
ORA C			10 110 001	2 6 1	B1
ORA D			10 110 010	2 6 2	B2
ORA E			10 110 011	2 6 3	B3
ORA H			10 110 100	2 6 4	B4
ORA L			10 110 101	2 6 5	B5
ORA M			10 110 110	2 6 6	B6
ORA A			10 110 111	2 6 7	B7
CMP	1	1	10 111 (SSS)	27(SSS)	
CMP B			10 111 000	2 7 0	B8
CMP C			10 111 001	2 7 1	B9
CMP D			10 111 010	2 7 2	BA
CMP E			10 111 011	2 7 3	BB
CMP H			10 111 100	2 7 4	BC

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
CMP L			10 111 101	2 7 5	BD
CMP M			10 111 110	2 7 6	BE
CMP A			10 111 111	2 7 7	BF

3．直接寻址指令

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
STA	3	4	00 110 010	062	32
LDA	3	4	00 111 010	072	3A
SHLD	3	5	00 100 010	042	22
LHLD	3	5	00 101 010	052	2A

F. 立即数指令

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
LXI	3	3	00 (rp)0 001	0(rp)1	

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
LXI B			00 000 001	0 0 1	01
LXI D			00 010 001	0 1 1	11
LXI H			00 100 001	0 2 1	21
LXI SP			00 110 001	0 3 1	31
MVI	2	2 or 3	00 (SSS) 110	0(SSS)6	
MVI B			00 000 110	0 0 6	06
MVI C			00 001 110	0 1 6	0E
MVI D			00 010 110	0 2 6	16
MVI E			00 011 110	0 3 6	1E
MVI H			00 100 110	0 4 6	26
MVI L			00 101 110	0 5 6	2E
MVI M			00 110 110	0 6 6	36
MVI A			00 111 110	0 7 6	3E
ADI	2	2	11 000 110	306	C6

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
ACI	2	2	11 001 110	316	CE
SUI	2	2	11 010 110	326	D6
SBI	2	2	11 011 110	336	DE
ANI	2	2	11 100 110	346	E6
XRI	2	2	11 101 110	356	EE
ORI	2	2	11 110 110	366	F6
CPI	2	3	11 111 110	376	FE

G. 转移指令

1. 跳转指令

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
PCHL	1	1	11 101 001	351	E9
JMP	3	3	11 000 011	303	C3
JC	3	3	11 011 010	332	DA

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
JNC	3	3	11 010 010	322	D2
JZ	3	3	11 001 010	312	CA
JNZ	3	3	11 000 010	302	C2
JM	3	3	11 111 010	372	FA
JP	3	3	11 110 010	362	F2
JPE	3	3	11 101 010	352	EA
JPO	3	3	11 100 010	342	E2

2．调用指令

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
CALL	3	5	11 001 101	315	CD
CC	3	3 or 5	11 011 100	334	DC
CNC	3	3 or 5	11 010 100	324	D4
CZ	3	3 or 5	11 001 100	314	CC

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
CNZ	3	3 or 5	11 000 100	304	C4
CM	3	3 or 5	11 111 100	374	FC
CP	3	3 or 5	11 110 100	364	F4
CPE	3	3 or 5	11 101 100	354	EC
CPO	3	3 or 5	11 100 100	344	E4

3．返回指令

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
RET	1	3	11 001 001	311	C9
RC	1	1 or 3	11 011 000	330	D8
RNC	1	1 or 3	11 010 000	320	D0
RZ	1	1 or 3	11 001 000	310	C8
RNZ	1	1 or 3	11 000 000	300	C0
RM	1	1 or 3	11 111 000	370	F8

助记符	字节	周期	二进制代码	八进制代码	十六进制代码
RP	1	1 or 3	11 110 000	360	F0
RPE	1	1 or 3	11 101 000	350	E8
RPO	1	1 or 3	11 100 000	340	E0

附录 B：在 Linux 上配置 Altair 8800

作者：**Kevin Cole (ubuntourist@hacdc.org) 2020.10.18**

这些操作说明将让用户能够在自己的 Linux 计算机上：

- 使用有意义的助记符（缩写）为机器指令编写 Altair 程序， 而不必向开关中拨入二进制数字；
- 将这些程序从 源代码文件**（有意义的助记符）汇编成可执行的 **目标代码文件，后者由本应被拨入的二进制数字组成；
- 生成 列表文件，这是一份同时显示源代码与相应二进制 机器代码的报告，并包含错误列表以及与源代码汇编相关的 其他有用信息；
- 在运行于其计算机上的 Altair 软件 模拟器 中测试代码；
- 在其 Linux 计算机与 Altair 之间建立通信；
- 向 Altair 发送可执行的目标代码文件；
- 以及，从其 Linux 计算机向 Altair 发送数据并接收返回的数据。

初学者应从运行 Raspberry Pi Desktop 操作系统的计算机开始。

软件包

首先，从安装 Macroassembler AS 开始。转到 `下载 页面``_ 并下载 **C** 源代码 的 ``最新测试版`_。 点击列表中第一行末尾的 **as bz2** 链接。

要构建并安装，你的系统还需要 `LaTeX`，它是 `texlive` 软件包的一部分。

> **备注**
>
> 1. 以下步骤假定你运行在 Linux 系统上，且该系统具备构建 该应用程序及其文档所需的全部程序。如果你运行的是 **Raspberry Pi Desktop** 操作系统，则一切正常， 可以放心继续。
> 2. 下面的 `make docs` 步骤会抱怨 `10` 或 `11` 次， 提示缺少 `german.sty` 文件。除非你需要德文文档， 否则可以安全地忽略该错误。每次抱怨时只需按 `ENTER`。

3. `make install``（截至 2021.09.13）引入了一个错误， 称无法创建目录 ``‘/usr/local/include/asl/.’`， 因为它已经存在，但该错误可以安全地忽略。（我已提交了一份修复，可能在你读到此处时已被接受。）

4. 如果你 没有 运行 Raspberry Pi Desktop 系统， 请参阅 `INSTALL` 文档，了解关于其他系统和定制的 更多烦琐、陈旧的细节。

5. 下面最后一步会移除损坏的德文文档。

按如下方式输入：

```
$ sudo apt update
$ sudo apt install texlive texlive-latex-extra
$ cd ~/Downloads
$ wget http://john.ccac.rwth-aachen.de:8000/ftp/as/source/c_version/
asl-current.tar.bz2
$ tar xjvf asl-current.tar.bz2
$ cd asl-current
$ cp -v Makefile.def-samples/Makefile.def-unknown-linux Makefile.def
$ make
$ make test
$ make docs
$ sudo make install
$ sudo rm /usr/local/doc/asl/as_DE.*
```

接下来，获取：

- 一款老式通信程序（`minicom`）、
- 一个用于 `minicom` 的文件传输协议附加组件（`gkermit`）、
- 一款 Altair 软件模拟器（`simh`），以及
- 大家都喜爱的页面浏览器（`most`）。见下文：它会派上用场。

你很可能可以跳过 **`gkermit`**，但它“完善”了 **`minicom`**，
后者默认希望手边有一份 Kermit 版本，即使 Altair 并不会用到它。

用以下命令将它们全部安装：

```
$ sudo apt update
$ sudo apt install minicom gkermit simh most
```

编写、汇编源代码

然后，使用你最喜欢的编辑器—即使它不是 `emacs` — 创建一个汇编器源文件，例如 `program.asm`。要 了解该源文件的语法，请研究：

- `Intel 8080 Programming Manual`_（PDF）
- **Macroassembler AS** 文档，提供以下形式：
  - PDF（/usr/local/doc/asl/as_EN.pdf）
  - HTML（/usr/local/doc/asl/as_EN.html）和
  - 纯文本（/usr/local/doc/asl/as_EN.doc）
- Intel 8080 Microcomputer Systems Users Manual，连同其他一些好东西，可从 Bitsavers archive 获取

例如，下面这段源代码是本书 Part 3, Section B 中通过 前面板拨入的那个加法程序的汇编语言版本。

```
;;;
```

```
        ORG     200o      ; 将程序计数器设置为地址 200
VAL1:   DB      5o        ; 地址 200 处的数据字节 = 5
VAL2:   DB      10o       ; 地址 201 处的数据字节 = 8（八进制 10）
SUM:    DB      0o        ; 地址 202 处的数据字节 = 0

        END               ; 结束
```

使用以下命令进行汇编：

```
$ asl -a -cpu 8080 -L -listradix 8 program.asm
```

这会生成一个列表报告（`program.lst`）和一个目标文件（`program.p`）。将生成的列表文件与手册中提供的八进制机器操作码进行比对。

不过，如果你已经安装了 `most`——你肯定想要的——用以下命令来汇编源文件会 **非常**有启发：

```
$ asl -a -cpu 8080 -L -listradix 16 program.asm
```

这会生成与第一条命令相同的二进制文件（`program.p`），但改变了列表文件的格式，以十六进制而非八进制来显示机器操作码，并且配合以下命令：

```
$ most program.p
```

你可以研究生成的实际二进制"可执行文件"，并通过在可执行文件中查找嵌入的机器操作码（十六进制）来"调试"它（与 `program.lst` 进行比对）。

无论用哪种方式汇编，你都会希望通过执行以下命令来生成 `program.bin` 和 `program.hex` 文件：

```
$ p2hex program.p
$ p2bin program.p
```

（用 `most` 查看 `.bin` 和 `.hex` 文件，并将其与你在 `.p` 和十六进制列表文件中看到的内容进行比对。）

在软件模拟器中运行你的程序

`simh` Debian 软件包可模拟至少 `30` 种不同的体系结构——其中包括除 Altair 之外我还曾有幸把玩过的另外七种。

每个模拟器都由单独的程序启动。Altair 模拟器被很合理地命名为 `altair`。要使用它：

1. 启动模拟器（`altair`）。

2. 如同实际机器的前面板那样，复位（`reset all`）。
3. 将你的二进制文件载入模拟器的“存储器”（`load program.bin`）。
4. 启动程序运行（`run`）。
5. 如果程序没有自行结束并停止，你需要 通过按下 `Ctrl-E``（将显示为 ``^E`）来停止模拟。
6. 用 `e` 后跟地址来检查存储单元，或 查阅并尝试 `help` 命令所展示的其他模拟器命令。
7. 完成后，输入 `exit`。

以上面的加法程序为例，输入以下内容：

```
$ altair

Altair 8800 simulator V3.8-1
sim> reset all
sim> load program.bin
131 Bytes loaded.
sim> run
^E

Simulation stopped, PC: 000004 (LDA 201)
sim> e 202
202:    015
sim> e PC,A,BC,DE,HL,C,Z,AC,P,S,SR,WRU
PC:     000004
...
sim> exit
Goodbye
$
```

上面第二条 `examine``（``e`）命令列出了寄存器以及 其他有用的位置。

simh 寄存器与状态标志

助记符	含义
PC	程序计数器
A	累加器
BC	寄存器对 BC

助记符	含义
DE	寄存器对 DE
HL	寄存器对 HL
C	进位标志
Z	零标志
AC	辅助进位标志
P	奇偶标志
S	符号标志
SR	前面板开关
BREAK	断点地址
WRU	中断字符

模拟器还有许多其他命令，但上面的内容足以完成 基本工作。`Ctrl-E` 会停止当前正在模拟器中运行的程序，并 报告当前的程序计数器（PC）以及停止时所执行的指令。

如果你运行的是上面所示的程序，**list file** 会告诉我们， 标有 **SUM** 的数据字节（DB）位于存储地址 `202` （八进制）。用模拟器的 `e` 命令检查该地址， 会发现其中包含数据值 `015`（八进制）。

> **备注**
>
> 你应该记住，上面的程序（来自 Part 3, Section B 的 **add two numbers** 程序）是一个无限循环，并且正在 非常快速地将代码一遍又一遍地重复执行。

当你通过按下 `Ctrl-E` 来停止模拟时，无法 得知当时正在执行哪条指令。这意味着程序计数器 **PC** 及其所指向的指令在每次运行模拟时都会变化—除非 你自己是一台超级计算机，并且拥有极快的反应速度。

在我上面的示例中，你可以看到程序计数器位于 `000004`，并且在模拟停止时正在执行指令 `LDA 201`， 但对你来说结果很可能不同。

上面那条 `131 Bytes loaded.` 是以十进制表示的。按八进制 计算，那就是 `203` 字节。你会注意到程序起始于存储地址 `000` （八进制），而加法运算的结果（即程序所使用的最后一个 存储字节）位于存储地址 `202`（八进制）。换句话说，程序 包含在存储器的前 `203`（八进制）个字节中—不过，在 存储器中字节 `015`（八进制）（即实际指令代码的最后一个字节） 与字节 `200`（八进制）（即待相加的第一个值的存放处）之间的 那些字节并未使用，因此这些字节是未初始化的， 并将包含不可预测的随机值。

`help` 命令所提供的更有用的模拟器命令汇总：

常用的 simh 命令

r{eset} {ALL|<device>}	复位模拟器
e{xamine} <list>	检查存储器或寄存器
ie{xamine} <list>	交互式检查存储器或寄存器
d{eposit} <list> <val>	在存储器或寄存器中存入
id{eposit} <list>	交互式在存储器或寄存器中存入
ev{aluate} <expr>	计算符号表达式
ru{n} {new PC}	复位并启动模拟

go {new PC}	启动模拟
s{tep} {n}	模拟 n 条指令

> **备注**
>
> `simh` Debian 软件包已经严重过时，如果你的 程序变得足够复杂—例如，试图将整个 CP/M 操作系统 载入模拟器—它会出错。更新的版本可以从 Altair clone site 和 GitHub repository 获取。

拆箱、插上电源并通电后，状态如下：

- inte
- prot
- **MEMR**
- inp
- **M1**
- out
- hlta
- stack
- **WO**
- int
- **WAIT**
- hlda

其中，大写、粗体表示处于“点亮”状态的发光二极管，小写表示处于“熄灭”状态的发光二极管。

将串口线缆一端插入 Altair 背面的 **TOP** 端口，另一端插入 Linux 机器的任意 USB 端口。然后执行 `lsusb` 命令。列表中应出现一个新的 USB 设备：：

```
Bus 003 Device 002: ID 067b:2303 Prolific Technology, Inc. PL2303 Serial Port
```

此外，插上线缆后，你应在 `/dev` 目录下看到两个新条目：一个 `/dev/ttyUSB0` 设备和一个 `/dev/serial/` 目录。我们要找的就是那个 `/dev/ttyUSB0`。

配置 MINICOM

首先，确认**你的**系统将这根线缆识别为 `/dev/ttyUSB0`。（这在不同电脑上可能不同，尤其是取决于你当时插了哪些设备。）我所知道的唯一的做法是：在插上线缆之前和之后各执行一次如下命令：：

```
$ ls /dev/tty*
```

并对比两次结果的差异。应当会多出一个 `/dev/ttyUSB0` 或 `/dev/ttyUSB1`—也可能是另一个编号，这取决于所插入的其他被识别为终端的 USB 设备的数量。[我也见过有些系统使用 `/dev/ttyACM0``（或 ``1`、`2` 等）来识别能够像终端一样通信的 USB 设备。]

> **备注**
>
> 如果你打算频繁地将 Altair 8800 克隆机连接到同一台电脑，或许值得配置一条 `udev` 规则。在 `/etc/udev/rules.d/` 中创建一个名为 99-local.rules 的文件，内容如下：：
>
> ```
> $ sudo cat > /etc/udev/rules.d/99-local.rules <<EOF # Written by Kevin Cole <ubuntourist@hacdc.org> 2020.10.19 (kjc) # Custom udev rules #
>
> # Create a symlink ALTAIR for Prolific Technology USB-Serial Controller # (idVendor=067b, idProduct=2303, driver=pl2303) # # Determined via: # # $ udevadm info -a -n /dev/ttyUSB0 | grep -i "driver" # SUBSYSTEMS=="usb",
>
>     DRIVERS=="pl2303", SYMLINK+="ALTAIR"
>
> EOF $ sudo udevadm control --reload-rules && sudo udevadm trigger
> ```
>
> 该 `serial` 属性可通过如下方式确定：：
>
> ```
> $ udevadm info -a -n /dev/ttyUSB0 | grep "[{]serial[}]"
> ```
>
> 并可用 `ATTRS{serial}=="..."` 替换 `DRIVERS--\"pl2303\"`，但由于每次将设备插入新的 USB 插槽时 serial 属性都会变化，使用驱动器标识（我希望）更具实用性。

`minicom` 在以一种合理的方式写入其配置文件这件事上，有着一种反常的幽默感。因此，为了让它把配置文件放到你想要的位置，人们不得不做一些愚蠢的折腾。你可以跟它较劲，或者直接编辑我的配置来满足你的需要。（参见下文的 **~/.minirc.altair** 一节。要与它较劲，按如下操作：：）

> $ sudo usermod -aG dialout kjcole $ cp /usr/share/doc/minicom/examples/minirc.dfl ~/.minirc.altair $ sudo minicom --setup altair Filenames and paths
>
> > A - Download directory: ~/Downloads B - Upload directory: ~/Code/Altair C - Script directory: D - Script program: runscript E - Kermit program: gkermit F - Logging options:
> >
> > > A - File name (empty=disable) : minicom.log B - Log connects and hangups : Yes C - Log file transfers : Yes
>
> **File transfer protocols:**
>
> > G kermit /usr/bin/gkermit -i -s H kermit /usr/bin/gkermit -i -r
>
> **Serial port setup**
>
> > A - Serial Device : /dev/ttyUSB0 B - Lockfile Location : /var/lock C - Callin Program : D - Callout Program : E - BPS/Parity/Bits : 115200 8N1 F - Hardware Flow Control : Yes G - Software Flow Control : No
>
> Save setup as altair Exit from Minicom $ cp /etc/minicom/minirc.altair ~/.minirc.altair $ sudo rm /etc/minicom/minirc.altair

> **备注**
>
> 如果你已成功创建了 `udev` 规则，那么应当将上述串行设备改用 `/dev/ALTAIR`，因为这样在多次重启之间会更加可靠。

此时，应当可以直接编辑 `~/.minirc.altair`，尽管关于该文件格式并没有任何文档说明。

~/.MINIRC.ALTAIR

编辑你的 `updir` 和 `downdir` 以适合自己的情况。（你可能不想试图把它放进 我 的主目录里。）

```
# Minicom Altair 8800 Clone configuration
# Last edited by Kevin Cole <ubuntourist@hacdc.org> 2020.10.18

pu pprog7           /usr/bin/gkermit -i -s
pu pprog8           /usr/bin/gkermit -i -r
```

```
pu port             /dev/ALTAIR
pu baudrate         9600
pu kermit           gkermit
pu updir            /home/kjcole/Code/Altair/
pu downdir          /home/kjcole/Downloads/
```

运行实测：在 Altair 克隆机上运行你的程序

一旦将上述内容放入你的主目录，现在就可以准备把代码装入 Altair 了：：

> $ minicom altair Welcome to minicom 2.7.1
>
> OPTIONS: I18n Compiled on Dec 23 2019, 02:06:26. Port /dev/ALTAIR, 08:32:24
>
> Press CTRL-A Z for help on special keys

现在，在 Altair 前面板上，抬起并保持 `STOP` 开关，在保持的同时，抬起并松开左侧的 `AUX` 开关，最后松开 `STOP`。你应当会在 `minicom` 窗口中看到：：

> === Configuration Monitor 1.94 ===
>
> 1. Floppies
> 2. PROMs
> 3. Serial Ports
> 4. Load .BIN
> 5. Load .HEX
>
> 6) Admin x) Exit to Altair
>
> Choice:

我发现载入 `.hex` 文件要稍微容易一些。载入 `.bin` 文件需要额外的步骤，告诉 Altair 从哪个地址开始载入文件。（`.hex` 文件已将这一信息内嵌其中。）

```
Choice: 5

Send .hex file now...
Ctrl-A S
```

`Ctrl-A` 后接 `S` 会弹出 `Send file` 对话框。

- 选择 `ascii` 作为文件传输协议
- `G``(``[Goto]`）后接从你的主目录到 `.hex` 文件的完整目录路径，以导航到该文件。例如，如果 `program.hex` 位于 `/home/kjcole/Code/Altair/Assembler`，则使用 ``Code/Altair/Assembler``（如果你已将 **upload** 目录配置为指向你存放所有 Altair 文件的目录，这一步就不需要了。）
- 键入 `S``(对应 ``[Show]`）以缩小文件名匹配范围，或滚动到相应文件并按 `SPACE`，该文件应被高亮；然后按 `O` 或 `ENTER``(对应 ``[Okay]`，该选项应当已被高亮）。

你应当会看到类似以下内容：：

```
+------------[ascii upload - Press CTRL-C to quit]------------+ |ASCII upload of "program.hex" | | |0.2 Kbytes transferred at 194 CPS... Done. | | | READY: press any key to continue... | | | | | +-------------------------------------------------------------+
```

- 按 `ENTER``(或者如提示所说，任意键），然后按 ``x` 以 **Exit to Altair**（退出到 Altair），如下所示：：

```
... 0219h bytes loaded at 0000h

=== Configuration Monitor 1.94 ===

1. Floppies

2. PROMs

3. Serial Ports

4. Load .BIN

5. Load .HEX

6) Admin x) Exit to Altair

Choice: x

Control Returned to Altair
```

最后，假设你在 `000` 处载入了程序，按下 Altair 克隆机上的 `RUN` 开关，你就应该“起跑”了。

要退出 `minicom` 并返回 `Bash` shell，键入 `Ctrl-A` 后接 `Q`。

MAKEFILE

MAKEFILE

一个 Makefile 可自动生成两份列表报告、目标文件、BIN 文件和 HEX 文件，甚至还能一次性汇编多个源文件。下面的 Makefile 接受以下命令：

`make`	汇编当前目录下全部 `asm` 文件
`make program`	汇编 `program.asm`
`make clean`	删除所有 `asm` 生成的全部文件

IMPORTANT：`make` 喜欢 **TAB** 而不是 **SPACE**。把下面代码中连续八个空格全部改成制表符！

```
#!/usr/bin/make -f
# Written by Kevin Cole <ubuntourist@hacdc.org> 2020.10.17 (kjc)
#
# This takes Altair-8800 assembler source files and produces:
#
#    - an object file                                  (.p)
#    - an Intel HEX file                               (.hex)
#    - a  binary executable                            (.bin)
#    - a  list report file with octal       op codes   (.oct.lst)
#    - a  list report file with hexadecimal op codes   (.oct.lst)
#    - an include(?) file -- thus far, empty           (.inc)
#
# To use:
#
#        make         # to generate for all .asm files
# or     make root    # (where "root" is the root of a specific
"root.asm")
# or     make clean   # to remove all generated files
#
#################################################################
#######

SHELL = /bin/bash
OBJECTS := $(patsubst %.asm,%,$(wildcard *.asm))

.PHONY : all
all : $(OBJECTS)
```

```
.PHONY : clean
clean :
        -rm -f *.hex         \
               *.bin         \
               *.lst         \
               *.inc         \
               *.p

% : %.asm
        asl -a -cpu 8080 -L -listradix  8 $<
        mv $@.lst $@.oct.lst
        asl -a -cpu 8080 -L -listradix 16 $<
        mv $@.lst $@.hex.lst
        p2hex $@.p
        p2bin $@.p
```

获取输入 / 显示输出

如果你为了读懂闪烁的指示灯并理解它们而伤透脑筋，而更想直接看到文本（或者，如果你希望能输入字符串，而不是每次拨动开关输入八位字符串），你就需要深入了解 **88-2SIO** 串行输入/输出硬件的机制，这在你迄今为止接触过的任何手册中都没有介绍。

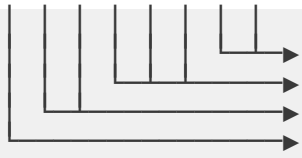
- 必须略读：`Altair 2SIO Serial I/O manual`_（PDF）
- 可选阅读：RS-232 Data Control Signals 和 UART

为了理解 Altair 与外部设备（如通过串行 / USB 线缆连接的终端）之间的通信，应当仔细阅读上述文档。它们提供的细节包括：

UART	通用异步接收/发送器
XMIT	发送（OUT）
RECV	接收（IN）
RTS	请求发送

以及同步（轮询）通信的设置：

```
15H = 0 0 0 1 0 1 0 1
    ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
```



```
1 = Clock Divide = 16
5 = 8 Data bits, No parity, 1 Stop bit (8N1)
0 = RTS = Low, XMIT interrupt = disabled
0 = RECV interrupt = disabled
```

提示

首先阅读 **Macroassembler AS** 手册中的 **第 6 章：实用程序**（第 251 页）。

基于 `add.asm` 生成的目标文件……

plist

`plist` 汇总 `.p` 文件的数据：

```
$ plist add.p
PLIST/C V1.42 Beta [Bld 174]
(C) 1992,2020 Alfred Arnold

Code-Type   Segment    Start-Address  Length (Bytes) End-Address
-------------------------------------------------------------------
8080/8085     CODE       00000000          000E       0000000D
8080/8085     CODE       00000080          0003       00000082
creator : AS 1.42 Beta [Bld 174]/x86_64-unknown-linux

altogether 17 bytes CODE
```

`plist` 能够分析目标文件并确定：

- 目标代码包含 Intel 8080 或 Intel 8085 的机器操作码。
- 一个 14 字节的代码块，应被载入存储单元 000 至 00D（十六进制），即 000 至 015（八进制）。
- 一个 3 字节的代码块应被载入 080 至 082（十六进制），即 200 至 202（八进制）。

p2bin

`p2bin` 将 `.p` 文件转换为二进制存储器转储，不带格式。默认情况下，未初始化的字节变为 `FF`。尽管加载可能会花费很长时间，但从表面上看，这似乎是最直接地呈现 Altair 存储器在程序载入后样貌的方式。

p2hex

`p2hex` 将 `.p` 目标文件转换为该文件的 ASCII 表示（`.hex`）。输出文件有若干种不同的格式可供选择，部分取决于目标机器。在我们的示例中：

```
$ p2hex add.p
$ most add.hex
:0E0000003A8000473A810080328200C300003F
:0300800005080070
:00000001FF
```

观察上述内容，并将其与汇编器生成的十六进制列表文件进行比较，我们可以确定前两行代表两个数据段：

Columns		Interpretation
Start	**End**	
1	1	冒号。记录起始分隔符
2	3	记录中的数据长度
4	7	存储器中数据的起始地址
8	9	？可能是记录类型？（00 或 01）
10	N-2	数据
N-1	N	？可能是记录校验和？

前两行在 8-9 列包含 00，而最后一行与列表文件所示的机器码似乎没有任何关系，它包含 01。也许是一个“数据结束”记录？

看哪，维基百科（理所当然地）有一条关于 Intel HEX 的词条，它确凿地证实：

- 8-9 是记录类型指示符，其中 `00` 表示**数据**，01 表示**文件结束**
- 最后两个字符是校验和

该词条还提供了一个 `checksum calculation algorithm`_（校验和计算算法）的示例。

Kermit 说明

在 `minicom` 中，发送和接收常用的默认 `kermit` 选项是 `-i -l %l -b %b`，根据 C-KERMIT 9.0 Unix man page and tutorial 的说明，其含义是：

`-i`	强制二进制（镜像）模式进行文件传输；
	隐含 `-V`
	等同于 `SET TRANSFER MODE MANUAL`，
	`SET FILE TYPE BINARY`
`-l %l`	在指定的串行通信上建立连接
	设备。
	等同于 `SET LINE``(``SET PORT`）命令
	参数：串行设备名称，例如 `/dev/ttyS0`
`-b %b`	串行设备的速度
	等同于 `SET SPEED`
	参数：每秒比特数的数值，用于串行连接

`gkermit` 似乎并不像我们曾经熟知的那样是一个完整的 Kermit，而仅仅是传输协议部分，并且依赖已建立的通信通道来确定上述的 `-l` 和 `-b`。因此，我已从自己的 `~/.minirc.altair` 配置中移除了这两个选项。

> 备注

哥伦比亚大学已不再是 Kermit 的归属地。它现在是 The Kermit Project。

补遗：快进到 2020

如前所述，*编译器*接受半可读的高级代码作为输入，并产生机器语言作为输出。

像 Arduino 这样的微处理器足够小、用途也足够单一，因此它们既不想要、 也不需要操作系统。然而，Raspberry Pi 足够复杂、功能足够强大，需要操作系统。

当有操作系统时，代码在执行大多数任务时都需要与操作系统协商。操作系统 就像交通警察、酒店预订代理，还有更多角色。除其他事项外，操作系统还必须：

- 告诉程序何时暂停、何时恢复，以便其他程序能够运行，
- 掌握可用磁盘空间的清单并进行合理分配，
- 监视来自程序的服务请求，并确定应由哪些程序来响应这些请求，
- 保护程序和数据，

等等。

当代码与硬件之间存在操作系统时，看似简单的键盘输入或屏幕输出就会变得 复杂。因此，为了使生成的机器码保持一定的可读性，下面的 `c` 程序并不 真正尝试输入或输出，因而也不会添加大量依赖于操作系统的代码。相反，它 模仿了 Altair 8800 操作手册中给出的第一个程序。

```
void main()
{
  char a =  5;
  char b = 10;

  a = a + b;
}
```

通过使用 `char` 而不是 `int`，取值被限制在 8 位，就像在 Altair 8800 上一样。

如果上面的程序保存为 `add.c`，就可以对其进行编译，并可以从编译器 生成的可执行目标代码文件产生汇编语言/机器语言清单。

重要细节：要理解所产生的汇编语言和机器码，你需要大致了解编译器所 面向的硬件。通常这是运行编译器的硬件，但并非总是如此。例如， **Arduino IDE** (集成开发环境) 运行在非 Arduino 处理器上，但产生的是 Arduino 机器码。

在次要程度上，了解操作系统内核的版本、编译器的版本以及所用汇编语言的 “方言”也很有用。

在下面的例子中，这些细节是：

处理器：	8 × Intel® Core™ i7-2960XM CPU @ 2.70GHz
内存：	31.3 GiB of RAM
操作系统类型：	Linux 64-bit
内核版本：	5.4.0-48-lowlatency
编译器：	GNU C Compiler (gcc) version 9.3.0

在 `Bash` 提示符下：

```
$ gcc -g -c add.c
$ objdump -drwC -Mintel add.o
```

（产生汇编语言/机器语言清单还有其他方法。例如，你可以让 GCC 在初始编译 期间生成它。不过，`objdump` 方法产生的输出更紧凑、更简洁。）

编译器接受源代码并产生*目标*文件。根据源代码以及编译方式的不同，目标 文件可能是函数和子程序的非可执行库（它们被*链接*到一个或多个主程序）， 也可能是完整的可执行程序。Linux 上的*共享目标库*（`.so`）和 Windows 上的*动态链接库*（`.dll`）就是前者的例子。例如，这样的库可能包含 数学算法的高效实现，或特殊的图形函数。

目标文件能够、而且在大多数系统上确实包含的不仅仅是机器语言。它们可以 包含一个“数据”段，其中保存着程序中使用的所有常量，以及有关在内存中 何处放置数据和可执行代码的信息。在 Linux（以及其他类 Unix 系统）上， 目标文件以 Executable and Linkable Format 格式存储，更常见的叫法是 它的缩写 **ELF**。

`objdump` 对二进制目标文件进行逆向工程，试图把它从不可打印的乱码 变回人类可读的形式—不过有些信息，比如原始变量名，可能会丢失。

`objdump` 对被要求做什么没有任何预设概念。命令中的选项 `-Mintel` 明确声明 `objdump` 应假定所使用的指令集是 **Intel**， 因此机器语言应被翻译回 Intel X86 architecture 助记符。

`objdump` 的输出如下：

```
add.o:     file format elf64-x86-64


Disassembly of section .text:
```

```
0000000000000000 <main>:
   0:        f3 0f 1e fa             endbr64
   4:        55                      push   rbp
   5:        48 89 e5                mov    rbp,rsp
   8:        c6 45 fe 05             mov    BYTE PTR [rbp-0x2],0x5
   c:        c6 45 ff 0a             mov    BYTE PTR [rbp-0x1],0xa
  10:        0f b6 55 fe             movzx  edx,BYTE PTR [rbp-0x2]
  14:        0f b6 45 ff             movzx  eax,BYTE PTR [rbp-0x1]
  18:        01 d0                   add    eax,edx
  1a:        88 45 fe                mov    BYTE PTR [rbp-0x2],al
  1d:        90                      nop
  1e:        5d                      pop    rbp
  1f:        c3                      ret
```

利用维基百科的 X86 architecture 条目，可以将上述内容解释为：

用到的寄存器

助记符	名称
`rbp`	基址指针
`rsp`	堆栈指针
`eax`	累加器（仅低 32 位）
`edx`	数据（仅低 32 位）
`al`	累加器（仅低 8 位）

1. `endbr64` 超出了本文档的范围。好奇的读者可以参考 StackOverflow 上的 问题 What does endbr64 instruction actually do?。就所有实际目的而言， 它在这段代码中就是一条 `nop`，可以安全地忽略。
2. 将 *基址指针*（`rbp`）的当前状态压栈保存。基址指针指向一个通常存放 数据（而非代码）的区域。
3. 将 堆栈指针*（``rsp``）的值移入基址指针（``rbp``）。我相信这是一种 编译器优化：由于数据量相对较小，且每个数据元素的尺寸很小，编译器 （我**认为*）选择把堆栈用作数据存储区，而不是通常分配给数据的内存 区域。堆栈比数据区域小，但访问速度更快。
4. 将值 5（`0x5`）移动到 `rbp - 2` 指向的存储单元。这就是 C 程序中的 `char a = 5;`。

5. 将值 10（0xa）移动到 rbp - 1 指向的位置。这对应于 C 程序中的 char b = 10;。

6. 将 a 移入 累加器*（``eax``）的低 32 位，将 ``b`` 移入 *数据寄存器 （edx）的低 32 位。尽管原始代码指定的是 8 位量，编译器却选择允许 最大 32 位的量。（例如，假设原始代码不是把 10 加到 5 上，而是计算 5 的 10 次方。5 和 10 两个量仍然放得进 8 位，但结果放不进去。）

7. 将数据寄存器（edx）的内容加到累加器寄存器（eax）的内容上。 赋值尚未完成，但这就是加法 a + b。结果存放在累加器中，但还没有 存储到内存里。

8. 只把累加器（al）的低 8 位移动到 rbp - 2，也就是第 3 步中存放 a 的位置。现在赋值已经完成（a = ...）。结合上一步， a = a + b; 现在已经完成。

9. nop 看起来没有必要，但编译器生成它大概是为了让 pop 语句落在 偶数编号的内存地址上。取决于硬件的架构和操作系统的特性，仔细的代码 字节对齐往往更高效，有时则是绝对必要的。

10. pop 通过将 rbp 重置回初始状态来清理内存。随后 rbp 就可以 被下一个程序使用。

11. ret。当你在 Bash 提示符下键入程序名来启动程序时，实际上就像调用 子程序一样在调用该程序。在代码的这个位置，return 将把控制权交还给 Bash（它是一个持续运行的程序，听候你的吩咐，并把它无法处理的任务分派给 其他程序）。

在 official Intel reference documents 中可以找到更详尽的介绍。不过， 十卷的篇幅有点让人吃不消。

产生汇编语言/机器语言清单还有其他方法，但上面是我能找到的最不啰嗦的方法。

Linux 上最流行的两个汇编器是 GNU Assembler`_（``gas`，又称 as）和 Netwide Assembler`_（``nasm`）。两者使用许多相同的助记符，产生相同的 机器码，但在语法上略有差异（例如，gas 用 # 作注释分隔符，而 nasm 用 ; 作注释分隔符）。

gas 已紧密融入 Linux 的体系结构，它是 GNU Compiler Collection （gcc，旧称 GNU C 编译器）的一部分，除 gas 外还包括 C、 FORTRAN、C++、Go 和 Java 等。

在类 Debian 系统上，gas 是 binutils 包的一部分。如果你还没有安装 gcc 包，我也建议安装它。nasm 则是独立的包。

你可以通过查看下面两个示例中的注释，用你选择的方言汇编并运行一个 “Hello World”（在确保已安装相应的包之后）：

"Hello World" 示例

汇编器	源代码
nasm	hello.asm

`gas`	hello.s

参见：

- Resources
- Comparison of assemblers

资源

如果你拥有一台 Altair 8800 克隆机，我推荐你阅读 `Altair Clone Operator's Manual``_ 以及 `Support`_ 页面上的其他资源。 尤其是 **PART 2 – CONFIGURATION MONITOR**，其中详细介绍了如何 轻松加载 ``.BIN`` 和 `.HEX` 文件。

Dave Dunfield 在他的 Classic Computing 页面上提供了大量 信息，我们就是从那里获得以下资料的：

- Assembly Language Programming System (A.L.P.S.)
- Device Management Facility (DMF-80/ST)

此外，Atari Clone 下载站的 Manuals 页面是以下资料的来源：

- 4kbootstrap.pdf
- Altair 2SIO Serial I-O.pdf
- Microsoft M80 Assembler.pdf

另外，Altair Front Panel 页面上也有不少有用的程序示例。

有位 Jeff Tranter 提供了一些关于 Macroassembler AS 的小贴士， 它需要从源代码编译，但显然能胜任工作。

服务

如果你的计算机出现问题，可以将其退回 MITS 维修。如果设备仍在保修 期内，任何有缺陷的部件都将免费更换。购机者需承担所有邮费。任何情况 下，都不得在外壳未完全装配好的情况下将设备寄回。

如果你因故需要将设备退回我们，请拆下计算机的顶盖，用胶带将各电路板 固定在各自的插座中，并在机箱顶部与电路板之间填满包装材料。盖好顶 盖，将设备装入坚固的纸箱，四周衬以厚厚一层包装材料。你可以使用碎报 纸、泡沫塑料或木丝。包装好的纸箱应用胶带整齐封好，并用结实的绳子捆 牢。务必在盒子外面用胶带贴上一封信，信中写明你的姓名、地址、故障描 述，以及原始发票（如果设备仍在保修期内）。

包裹可通过邮政包裹或 UPS–邮寄；如需更快的服务，可通过航空邮政包 裹寄送。务必为包裹投保。

邮寄至：	MITS, Inc. 6328 Linn Ave. N.E. Albuquerque, New Mexico 87108

如果对机器的基本设计进行了任何更改，或以任何方式动过机器的内部结 构，所有保修均告失效。

MITS ALTATR 8800 价目表

1976 年 1 月 1 日

零件编号	说明	套件	组装	交货天数
8800	Altair 8800 计算机	$ 439.00	$ 621.00	60
COMTER II	配音频磁带 I/O 的终端	780.00	920.00	60
CT-256	Comter 256 终端	745.00	885.00	45-60
CT257,8 or 9	CT-256 的第 2、3、4 页	95.00	105.00	45-60
CT-8096	CRT 终端	TBD	TBD	TBD
88-VLCT	低成本终端	129.00	169.00	45-60
88-80LP	行式打印机与控制器	1,750.00	1,975.00	60
88-TTY	Teletype ASR-33	1,500.00	1,500.00	60
88-MM	为 88-MCS 增加 256 字	14.00	26.00	30
88-1MCS	1K 静态存储器	97.00	139.00	30
88-4MCD	4K 动态存储器	195.00	275.00	60

零件编号	说明	套件	组装	交货天数
88~DCDD	磁盘控制器与 1 台驱动器	1,480.00	1,980.00	60
88-DISC	磁盘驱动器（含机箱）	1,180.00	1,600.00	60
88-DMAC	直接存储器访问控制器	98.00	149.00	TBD
88-DMAE	直接存储器 I/O 通道	126.00	186.00	TBD
88-DMAI	直接存储器 I/O 通道	123.00	183.00	TBD
88-4PIO	4 端口并行 I/O	86.00	112.00	30
88-PP	4PIO 的附加端口	30.00	39.00	30
88-2S10	2 端口串行板（状态 I/O）	115.00	144.00	30
88-SP	2SI0 板的附加端口	24.00	35.00	30
88-EC	仅扩展器母板	16.00	31.00	30
88-MB	88-EC，含连接器与插卡导轨	65.00	138.00	30
88-EBC	扩展器机箱	394.00	485.00	60
88-EXC	延长卡	57.00	83.00	30
88-ACR	音频磁带录音接口	128.00	174.00	30

零件编号	说明	套件	组装	交货天数
88-VI	向量中断	126.00	179.00	90
88-RTC	实时时钟	53.00	84.00	90
88-PPCB	原型 PC 电路板	57.00	84.00	30
88-FAN	冷却风扇	16.00	20.00	30
88-FMC	PROM 存储卡（不含 PROM）	65.00	128.00	60
88-PROM	PROM (256 x 8 字节)	25.00	37.00	60
88-PPC	PROM 编程器卡		联系工厂	90
25DB	机箱用 I/O 插座	11.00	25.00	30
MS-416	MITScope-4 通道示波器	127.00	189.00	30
680F	680 MPU 单元（组装状态 I/O）	345.00	420.00	60
680T	680，无前面板	280.00		60
680	CPU 板（带微处理器芯片）	195.00	275.00	60
680	PROM 256 x 8 位 PROM	25.00	37.00	60
680FAN	Peewee 风扇选件	16.00	20.00	60

零件编号	说明	套件	组装	交货天数
680Sacket	680 IC 插座选件	29.00	42.00	60

注：价格、规格、研发和交货时间如有变更，恕不另行通知。

8800 系统建议价格	套件	组装	交货 天数
System I | ALTAIR Basic I	1,712.00	7,265.00	60
System II | ALTAIR Extendod Basic II	1,893.00	2,566.00	60
System III | ALTAIR DOS/Basic III	4,714.00	6,397.00	90
System IV | ALTAIR Extended Engr/Acctg IV	7,938.00	9,985.00	120

（若要以 Teletype 替换 COMTER II，套件价格加收 720.00 美元，组装价格 加收 580.00 美元。）

整套系统的邮费与手续费将另行报价。

8800 系统软件

		已购 8800 的用户 加购以下配置的价格：		
4K Basic	$150.00	4K 存储器、I/O	$ 60.00	30
8K Basic	$200.00	含 8K 存储器、I/O	$ 75.00	30
EXT Basic	$350.00	含 12K 存储器、I/O	$ 150.00	30
	$175.00		$ 30.00	30

		已购 8800 的用户 加购以下配置的价格：		
Package #1		含 8K 存储器、I/O		
DOS	$500.00	含 12K 存储器、I/O	$ 150.00	60
DEBUG	$100.00	4K 存储器、I/O	$ 25.00	60

上述软件的更新副本或第二份副本收取 15.00 拷贝费。对于更新软件的
用户，除更新费外还将加收拷贝费。

订购软件时请注明需要纸带还是音频磁带，DOS 除外—它仅以磁盘形式提供。

手册

8800 系统软件

CT-256		
	操作手册	6.50
	组装手册	10.00
	工作原理	10.00

8800		
	操作手册	7.50
	组装手册	9.00
	工作原理	9.00

8800		
	特别优惠-活页装订的全部三本手册	$15.00

680		
	操作手册	7.50
	组装手册	7.50
	工作原理	10.00
	特别优惠-活页装订的全部三本手册	$14.50

- BASIC 语言文档 10.00

邮费与条款

条款：订货时付现，可使用 Mastercharge 或 Bankamericard。

邮费与手续费：

1. 每台终端、计算机、行式打印机和磁盘加收 8.00 美元。
2. 模块板另收费用
    1. 与计算机一同订购时为 -0-
    2. 单独订购时为 3.00 美元
3. 手册价格已含邮费。
4. Teletype 订单将以卡车货运寄送，运费到付。
5. 加拿大、夏威夷及阿拉斯加的邮费需另行报价。

上述规定仅适用于美国国内货运。除非另有说明，海外货运通常经由我们的 货运代理 Emery Airfreight 以航空运费到付的方式发运。

索引和表格

- 索引
- 模块索引
- 搜索页面

导航

- Altair 8800 操作手册 v2.0 »
- Altair 8800 操作手册 v2.0

© Copyright 1975, MITS, Inc..
Created using Sphinx 9.2.0.